Hindman's finite sums and products: a machine-checked derivation of Corollary 1.2 from Theorem 1.1

Abstract

We present a Lean 4/Mathlib formalization of Corollary 1.2 of the source paper. It takes Hindman's finite sums and products conjecture, Theorem 1.1, as a hypothesis, including the growth conditions on the monochromatic sequence. From this hypothesis the corollary follows: for every finite colouring of the natural numbers and every $m \geq 1$ there is an m-element set A of positive integers such that $\mathrm{FS}(A) \cup \mathrm{FP}(A)$ is monochromatic, $|\mathrm{FS}(A)| = |\mathrm{FP}(A)| = 2^m - 1$ and $\mathrm{FS}(A) \cap \mathrm{FP}(A) = A$. Theorem 1.1 itself is not formalized. It enters only as a hypothesis whose content is spelled out in this report. The formal proof uses Theorem 1.1 with $R = 2$, $D = 1$ and then argues purely about finite sets satisfying a lacunarity condition.

Source	openai/math@fd4aeeb, family 164, OpenAIMath_164_HindmanDistinctExpressions
Lean	leanprover/lean4:v4.23.0, Mathlib 37df177aaa
Machine checks	compiles; no `sorry`; statement matches target; module build
Axioms	`propext`, `Classical.choice`, `Quot.sound`
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Compute	26 AIProver sessions in 18 jobs, 46 session-hours
Hosted models	10 calls to claude-opus-5-5, claude-sonnet-5-5, $1.07 up to the final review
Run	`srv_20261008_212508_OpenAIMath_164_HindmanDistinctExpressions`

Contents

1 Statement **1**

2 Formalization **2**

3 Proof **4**

3.1 Overview 4

3.2 Proof of the theorem 23

4 Independent review **24**

A Complete Lean file **26**

1 Statement

Let $\mathbb{N} = \{1, 2, \ldots\}$ and $[r] = \{1, \ldots, r\}$. For a finite set $A \subset \mathbb{N}$ put

$$\mathrm{FS}(A) = \Big\{ \sum_{a \in B} a : \ \varnothing \neq B \subseteq A \Big\}, \qquad \mathrm{FP}(A) = \Big\{ \prod_{a \in B} a : \ \varnothing \neq B \subseteq A \Big\}.$$

Theorem 1.1 (Theorem 1.1 of the source; used here as a hypothesis)**.** *Let $r, m \geq 1$ be integers and let $R \geq 2$ and $D \geq 1$ be real numbers. For every colouring $\chi : \mathbb{N} \to [r]$ there are positive integers $a_1 < \cdots < a_m$ and a colour $c \in [r]$ such that*

$$\chi\Big(\sum_{j\in J} a_j\Big) = \chi\Big(\prod_{j\in J} a_j\Big) = c \quad \text{for every } \varnothing \neq J \subseteq [m].$$

Moreover,

$$a_1 > R, \qquad a_d > R\Big(\sum_{k<d} a_k + \prod_{k<d} a_k\Big)^D \quad (2 \leq d \leq m).$$

Theorem 1.2 (Corollary 1.2)**.** *Let χ be a finite colouring of $\mathbb{N}$ and let $m \geq 1$. Then there is an m-element set $A \subset \mathbb{N}$ with the following properties:*

- $\mathrm{FS}(A) \cup \mathrm{FP}(A)$ *is monochromatic;*
- $|\mathrm{FS}(A)| = |\mathrm{FP}(A)| = 2^m - 1$*;*
- $\mathrm{FS}(A) \cap \mathrm{FP}(A) = A$*.*

2 Formalization

Lean's natural numbers are $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$. Throughout we write $\mathbb{N}_0$ for them and keep $\mathbb{N}$ for the positive integers. For a finite set $B \subseteq \mathbb{N}_0$ write

$$\sigma(B) = \sum_{y\in B} y, \qquad \pi(B) = \prod_{y\in B} y,$$

with the conventions $\sigma(\varnothing) = 0$ and $\pi(\varnothing) = 1$. No library declarations beyond Mathlib are used.

Definition 2.1 (FS)**.** For a finite set $A \subseteq \mathbb{N}_0$, $\mathrm{FS}(A)$ is the finite set $\{\sigma(B) : B \subseteq A,\ B \neq \varnothing\}$. It is formed as the image of the family of nonempty subsets of A under σ.

Lean: `FS`

```
/-- FS(A): the set of all sums over nonempty subsets of A. -/
def FS (A : Finset ℕ) : Finset ℕ :=
  (A.powerset.filter (fun B => B.Nonempty)).image (fun B => ∑ a ∈ B, a)
```

Definition 2.2 (FP)**.** For a finite set $A \subseteq \mathbb{N}_0$, $\mathrm{FP}(A)$ is the finite set $\{\pi(B) : B \subseteq A,\ B \neq \varnothing\}$. It is formed as the image of the family of nonempty subsets of A under π.

Lean: `FP`

```
/-- FP(A): the set of all products over nonempty subsets of A. -/
def FP (A : Finset ℕ) : Finset ℕ :=
  (A.powerset.filter (fun B => B.Nonempty)).image (fun B => ∏ a ∈ B, a)
```

These agree with the source definitions whenever $A \subset \mathbb{N}$. The formal definitions also accept sets containing 0.

Definition 2.3 (The proposition `Thm11`)**.** `Thm11` is the following proposition. For all integers $r, m \geq 1$, all reals $R \geq 2$ and $D \geq 1$, and every map $\chi : \mathbb{N}_0 \to \{0, \dots, r-1\}$, there exist $a : \{0, \dots, m-1\} \to \mathbb{N}_0$ and $c \in \{0, \dots, r-1\}$ with the following properties:

1. a is strictly increasing and $a_j > 0$ for all j;
2. for every nonempty $J \subseteq \{0, \dots, m-1\}$,
$$\chi\Big(\sum_{j \in J} a_j\Big) = c \quad \text{and} \quad \chi\Big(\prod_{j \in J} a_j\Big) = c;$$
3. $R < a_0$;
4. for every d with $1 \leq d \leq m-1$,
$$R\Big(\sum_{k<d} a_k + \prod_{k<d} a_k\Big)^D < a_d.$$

The power in (4) is the real power of a nonnegative real number.

Lean: `Thm11`

```
/-- Theorem 1.1 (Hindman's FS+FP conjecture with growth conditions), stated as a
proposition so that it can be used as a hypothesis. Indices run over `Fin m`, so
`a ⟨0, _⟩` plays the role of `a_1`. -/
def Thm11 : Prop :=
  ∀ (r m : ℕ), 1 ≤ r → 1 ≤ m → ∀ (R D : ℝ), 2 ≤ R → 1 ≤ D →
    ∀ χ : ℕ → Fin r, ∃ (a : Fin m → ℕ) (c : Fin r),
      StrictMono a ∧ (∀ j, 0 < a j) ∧
      (∀ J : Finset (Fin m), J.Nonempty →
        χ (∑ j ∈ J, a j) = c ∧ χ (∏ j ∈ J, a j) = c) ∧
      (∀ d : Fin m, d.val = 0 → R < (a d : ℝ)) ∧
      (∀ d : Fin m, 1 ≤ d.val →
        R * ((((∑ k ∈ Finset.Iio d, a k : ℕ) : ℝ)
              + (((∏ k ∈ Finset.Iio d, a k : ℕ) : ℝ)) ^ D) < (a d : ℝ))
```

The index 0 plays the role of a_1 in the source, and the colours are labelled $0, \dots, r-1$. The colouring is defined on $\mathbb{N}_0$ rather than on $\mathbb{N}$. This makes no difference in content:

- every colouring of $\mathbb{N}$ extends to $\mathbb{N}_0$, and every colouring of $\mathbb{N}_0$ restricts to $\mathbb{N}$;
- all values at which χ is evaluated in the conclusion are sums or products of the positive a_j, hence positive.

Thus `Thm11` is a faithful transcription of Theorem 1.1. It is not proved; it appears only as a hypothesis of the main result.

The following two facts unfold the definitions of FS and FP into membership criteria. They are immediate from the definition of the image of a finite set.

Lemma 2.4 (`setting_mem_FS`)**.** *For a finite $A \subseteq \mathbb{N}_0$ and $x \in \mathbb{N}_0$: $x \in \mathrm{FS}(A)$ if and only if there is a nonempty $B \subseteq A$ with $\sigma(B) = x$.*

Lean: setting_mem_FS

```
theorem setting_mem_FS (A : Finset ℕ) (x : ℕ) :
    x ∈ HindmanFSFP.FS A ↔ ∃ B : Finset ℕ, B ⊆ A ∧ B.Nonempty ∧ ∑ a ∈ B, a = x := by
  simp only [HindmanFSFP.FS, Finset.mem_image, Finset.mem_filter, Finset.mem_powerset, and_assoc]
```

Lemma 2.5 (setting_mem_FP)**.** *For a finite $A \subseteq \mathbb{N}_0$ and $x \in \mathbb{N}_0$: $x \in \mathrm{FP}(A)$ if and only if there is a nonempty $B \subseteq A$ with $\pi(B) = x$.*

Lean: setting_mem_FP

```
theorem setting_mem_FP (A : Finset ℕ) (x : ℕ) :
    x ∈ HindmanFSFP.FP A ↔ ∃ B : Finset ℕ, B ⊆ A ∧ B.Nonempty ∧ ∏ a ∈ B, a = x := by
  simp only [HindmanFSFP.FP, Finset.mem_image, Finset.mem_filter, Finset.mem_powerset, and_assoc]
```

The target statement. Lean: target statement openaimath_164_hindmandistinctexpressions, fixed before the proof; solution is checked to have exactly this type

```
theorem openaimath_164_hindmandistinctexpressions (hT : HindmanFSFP.Thm11)
    (r m : ℕ) (hr : 1 ≤ r) (hm : 1 ≤ m) (χ : ℕ → Fin r) :
    ∃ A : Finset ℕ, A.card = m ∧ (∀ a ∈ A, 0 < a) ∧
      (∃ c : Fin r, ∀ x ∈ HindmanFSFP.FS A ∪ HindmanFSFP.FP A, χ x = c) ∧
      (HindmanFSFP.FS A).card = 2 ^ m - 1 ∧ (HindmanFSFP.FP A).card = 2 ^ m - 1 ∧
      HindmanFSFP.FS A ∩ HindmanFSFP.FP A = A
```

The target statement is the statement proved as solution. Assume Thm11, and let integers $r, m \geq 1$ and a colouring $\chi : \mathbb{N}_0 \to \{0, \ldots, r-1\}$ be given. Then there is a finite set $A \subseteq \mathbb{N}_0$ such that:

1. $|A| = m$, and every element of A is positive (so $A \subset \mathbb{N}$);
2. there is a colour c with $\chi(x) = c$ for all $x \in \mathrm{FS}(A) \cup \mathrm{FP}(A)$;
3. $|\mathrm{FS}(A)| = 2^m - 1$ and $|\mathrm{FP}(A)| = 2^m - 1$;
4. $\mathrm{FS}(A) \cap \mathrm{FP}(A) = A$.

The hypothesis Thm11 stands for Theorem 1.1 of the source, and χ stands for the given finite colouring with r colours. The subtraction $2^m - 1$ is truncated subtraction on $\mathbb{N}_0$, which is ordinary subtraction because $2^m \geq 1$. The statement is therefore exactly Corollary 1.2, conditional on Theorem 1.1, as in the source.

3 Proof

3.1 Overview

The following condition replaces the indexed growth conditions. It is not a Lean declaration; it abbreviates the hypothesis that recurs in the lemmas below.

Definition 3.1 (Condition (G))**.** For a finite set $A \subseteq \mathbb{N}_0$ and $x \in \mathbb{N}_0$ write $A_{<x} = \{y \in A : y < x\}$. We say A satisfies *(G)* if

$$2\big(\sigma(A_{<x}) + \pi(A_{<x})\big) < x \qquad \text{for every } x \in A.$$

The formal proof follows the source proof, organized as follows.

- **Extracting the sequence.** Lemma 3.2 applies Theorem 1.1 with $R = 2$, $D = 1$. This yields the inequality $2\big(\sum_{k<d} a_k + \prod_{k<d} a_k\big) < a_d$ for every index d, including $d = 0$.
- **Passing to the set A.** Lemmas 3.3 and 3.4 transfer the growth and colour information from the sequence to its range $A = \{a_0, \dots, a_{m-1}\}$. Growth becomes condition (G), and the colour statement becomes monochromaticity on all nonempty $B \subseteq A$.
- **Elementary consequences of (G).**
  - Lemma 3.5: every element exceeds 2.
  - Lemma 3.6: $2(\sigma(B) + \pi(B)) < x$ whenever all elements of $B \subseteq A$ are below $x \in A$.
  - Lemma 3.7: an element of B is at most $\sigma(B)$ and at most $\pi(B)$.
- **Injectivity.** Lemmas 3.10 and 3.11 are the source's injectivity of the subset maps. Both cancel the common part and look at the largest element of the symmetric difference. The product case goes through the disjoint version Lemma 3.9.
- **Sum versus product.** Lemma 3.12 is the source's comparison of a subset sum with a subset product.
- **Counting and assembly.**
  - Lemmas 3.13 to 3.15 turn injectivity into the cardinality statements.
  - Lemma 3.16 gives the intersection.
  - The final subsection assembles everything.

Deviations from the source proof.

- The source uses only that each a_j exceeds the sum and the product of its predecessors and that all elements are at least 2. The formal proof uses the stronger condition (G), which carries the factor 2 coming from $R = 2$. With this condition, the bound "the sum lies in $[a_j, 2a_j)$" becomes $\sigma(B) < \frac{3}{2}b < 2b$ for the largest element b of B. The formal proof also obtains that every element exceeds 2, not merely that it is at least 2.
- All arguments after Lemma 3.4 are phrased for an abstract finite set satisfying (G), using largest elements in place of greatest indices.
- Lemma 3.8 (the disjoint version of sum-injectivity) is proved but not used: Lemma 3.10 repeats the argument directly on the symmetric difference.
- Lemma 3.15 redoes the count of nonempty subsets inline instead of citing Lemma 3.13.

Lemma 3.2 (`hfs_extract`)**.** *Assume* `Thm11`*. Let $r, m \geq 1$ and $\chi : \mathbb{N}_0 \to \{0, \dots, r-1\}$. Then there exist $a : \{0, \dots, m-1\} \to \mathbb{N}_0$ and a colour c with the following properties:*

1. *a is strictly increasing and $a_j > 0$ for all j;*
2. *$\chi(\sum_{j\in J} a_j) = c$ and $\chi(\prod_{j\in J} a_j) = c$ for every nonempty $J \subseteq \{0,\dots,m-1\}$;*
3. *for every $d \in \{0,\dots,m-1\}$,*
$$2\Big(\sum_{k<d} a_k + \prod_{k<d} a_k\Big) < a_d.$$

Lean: `hfs_extract` — proved by AIProver, job 1, session 0

```
theorem hfs_extract (hT : HindmanFSFP.Thm11) (r m : ℕ) (hr : 1 ≤ r) (hm : 1 ≤ m)
    (χ : ℕ → Fin r) :
    ∃ (a : Fin m → ℕ) (c : Fin r), StrictMono a ∧ (∀ j, 0 < a j) ∧
      (∀ J : Finset (Fin m), J.Nonempty → χ (∑ j ∈ J, a j) = c ∧ χ (∏ j ∈ J, a j) = c) ∧
      (∀ d : Fin m, 2 * (∑ k ∈ Finset.Iio d, a k + ∏ k ∈ Finset.Iio d, a k) < a d) := by

  have h := hT r m hr hm (2 : ℝ) (1 : ℝ) (by norm_num) (by norm_num) χ
  rcases h with ⟨a, c, hmono, hpos, hcolor, hfirst, hrest⟩
  refine ⟨a, c, hmono, hpos, hcolor, ?_⟩
  intro d
  by_cases hd0 : d.val = 0
  ·
    have hineq : (2 : ℝ) < (a d : ℝ) := hfirst d hd0
    have hempty : Finset.Iio d = (∅ : Finset (Fin m)) := by
      apply Finset.eq_empty_iff_forall_notMem.mpr
      intro x hx
      have hxval := Finset.mem_Iio.mp hx
      have hxval_lt : x.val < d.val := hxval
      rw [hd0] at hxval_lt
      exact Nat.not_lt_zero _ hxval_lt
    have hsum : (∑ k ∈ Finset.Iio d, a k) = 0 := by simp [hempty]
    have hprod : (∏ k ∈ Finset.Iio d, a k) = 1 := by simp [hempty]
    rw [hsum, hprod]

    have hcast : ((2 : ℕ) : ℝ) < (a d : ℝ) := by simpa using hineq

    simpa [Nat.cast_add, Nat.cast_mul, Nat.cast_one] using (Nat.cast_lt (α := ℝ)).mp hcast
  ·
    have hdpos : 1 ≤ d.val := Nat.one_le_of_lt (Nat.pos_of_ne_zero hd0)
    have hineq : (2 : ℝ) * (((∑ k ∈ Finset.Iio d, a k : ℕ) : ℝ) + ((∏ k ∈ Finset.Iio d, a k : ℕ) : ℝ))
↪   ^ (1 : ℝ) < (a d : ℝ) :=
      hrest d hdpos

    have hineq' : (2 : ℝ) * (((∑ k ∈ Finset.Iio d, a k : ℕ) : ℝ) + ((∏ k ∈ Finset.Iio d, a k : ℕ) : ℝ))
↪   < (a d : ℝ) := by
      simpa [Real.rpow_one] using hineq

    have hcast : ((2 * ((∑ k ∈ Finset.Iio d, a k) + (∏ k ∈ Finset.Iio d, a k)) : ℕ) : ℝ) < (a d : ℝ) := by
      simpa [Nat.cast_add, Nat.cast_mul] using hineq'
    exact (Nat.cast_lt (α := ℝ)).mp hcast
```

Proof. Apply `Thm11` with $R = 2$ and $D = 1$. This gives a and c satisfying (1) and (2), and it remains to check (3).

Case $d = 0$. The index set $\{k < 0\}$ is empty, so the sum is 0 and the product is 1. Inequality (3) reads $2(0 + 1) < a_0$, which is the condition $2 < a_0$ supplied by `Thm11`.

Case $d \geq 1$. Since $x^1 = x$ for real powers, the hypothesis reads

$$2\Big(\sum_{k<d} a_k + \prod_{k<d} a_k\Big) < a_d$$

as an inequality of reals between images of natural numbers. It therefore holds in $\mathbb{N}_0$. □

Lemma 3.3 (`hfs_growth_image`)**.** *Let $a : \{0, \dots, m-1\} \to \mathbb{N}_0$ be strictly increasing, and suppose*

$$2\Big(\sum_{k<d} a_k + \prod_{k<d} a_k\Big) < a_d \quad \textit{for every } d.$$

Then the range $A = \{a_0, \dots, a_{m-1}\}$ satisfies (G).

Lean: `hfs_growth_image` — proved by AIProver, job 1, session 0

```
theorem hfs_growth_image (m : ℕ) (a : Fin m → ℕ) (ha : StrictMono a)
    (hg : ∀ d : Fin m, 2 * (∑ k ∈ Finset.Iio d, a k + ∏ k ∈ Finset.Iio d, a k) < a d) :
    ∀ x ∈ Finset.univ.image a,
      2 * (∑ y ∈ (Finset.univ.image a).filter (fun y => y < x), y
        + ∏ y ∈ (Finset.univ.image a).filter (fun y => y < x), y) < x := by
  intro x hx
  rcases Finset.mem_image.1 hx with ⟨d, _, hd⟩
  subst hd
  have h_filter_eq : (Finset.univ.image a).filter (fun y => y < a d) =
      (Finset.Iio d).image a := by
    apply Finset.Subset.antisymm
    · intro y hy
      rcases Finset.mem_filter.1 hy with ⟨hy_img, hy_lt⟩
      rcases Finset.mem_image.1 hy_img with ⟨i, _, hi⟩
      have hi_lt_d : i < d := by
        by_contra! hge
        have hle := ha.monotone hge
        rw [hi] at hle
        linarith
      apply Finset.mem_image.mpr
      exact ⟨i, Finset.mem_Iio.mpr hi_lt_d, hi⟩
    · intro y hy
      rcases Finset.mem_image.1 hy with ⟨i, hi, rfl⟩
      have hi_lt_d : i < d := Finset.mem_Iio.1 hi
      have ha_lt : a i < a d := ha hi_lt_d
      apply Finset.mem_filter.mpr
      exact ⟨Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩, ha_lt⟩
  have h_sum_eq : ∑ y ∈ (Finset.univ.image a).filter (fun y => y < a d), y =
      ∑ k ∈ Finset.Iio d, a k := by
    rw [h_filter_eq, Finset.sum_image]
    intro i _ j _ h
    apply ha.injective h
  have h_prod_eq : ∏ y ∈ (Finset.univ.image a).filter (fun y => y < a d), y =
      ∏ k ∈ Finset.Iio d, a k := by
    rw [h_filter_eq, Finset.prod_image]
    intro i _ j _ h
    apply ha.injective h
  rw [h_sum_eq, h_prod_eq]
  exact hg d
```

Proof. Let $x \in A$, say $x = a_d$. We claim $A_{<a_d} = \{a_k : k < d\}$.

- If $k < d$ then $a_k < a_d$ by strict monotonicity, so $a_k \in A_{<a_d}$.
- Conversely, let $a_i < a_d$. If $i \geq d$ we would have $a_i \geq a_d$ by monotonicity, a contradiction. Hence $i < d$.

Since a is injective, summing or multiplying over $\{a_k : k < d\}$ is the same as summing or multiplying a_k over $k < d$. Therefore

$$\sigma(A_{<x}) = \sum_{k<d} a_k, \qquad \pi(A_{<x}) = \prod_{k<d} a_k,$$

and the hypothesis at d is exactly (G) at x. □

Lemma 3.4 (`hfs_mono_image`)**.** *Let the following data be given:*

- *an injective map $a : \{0, \dots, m-1\} \to \mathbb{N}_0$;*
- *a colouring $\chi : \mathbb{N}_0 \to \{0, \dots, r-1\}$ and a colour c;*
- *the hypothesis that $\chi(\sum_{j\in J} a_j) = c$ and $\chi(\prod_{j\in J} a_j) = c$ for every nonempty $J \subseteq \{0, \dots, m-1\}$.*

Then $\chi(\sigma(B)) = c$ and $\chi(\pi(B)) = c$ for every nonempty $B \subseteq \{a_0, \dots, a_{m-1}\}$.

Lean: `hfs_mono_image` — proved by AIProver, job 1, session 0

```
theorem hfs_mono_image (m r : ℕ) (a : Fin m → ℕ) (χ : ℕ → Fin r) (c : Fin r)
    (ha : Function.Injective a)
    (hc : ∀ J : Finset (Fin m), J.Nonempty → χ (∑ j ∈ J, a j) = c ∧ χ (∏ j ∈ J, a j) = c) :
    ∀ B ⊆ Finset.univ.image a, B.Nonempty → χ (∑ x ∈ B, x) = c ∧ χ (∏ x ∈ B, x) = c := by
  intro B hBsub hBnon

  let J := Finset.univ.filter fun j : Fin m => a j ∈ B
  have hJnon : J.Nonempty := by
    rcases hBnon with ⟨x, hx⟩
    have hx_image : x ∈ Finset.univ.image a := hBsub hx
    rcases Finset.mem_image.mp hx_image with ⟨j, hj, rfl⟩
    refine ⟨j, Finset.mem_filter.mpr ⟨Finset.mem_univ _, hx⟩⟩
  have h_image_eq : Finset.image a J = B := by
    apply Finset.Subset.antisymm
    · intro x hx
      rcases Finset.mem_image.mp hx with ⟨j, hj, rfl⟩
      rcases Finset.mem_filter.mp hj with ⟨_, hjB⟩
      exact hjB
    · intro x hx
      have hx_image : x ∈ Finset.univ.image a := hBsub hx
      rcases Finset.mem_image.mp hx_image with ⟨j, hj, rfl⟩
      apply Finset.mem_image.mpr
      exact ⟨j, Finset.mem_filter.mpr ⟨Finset.mem_univ _, hx⟩, rfl⟩
  have hsum : ∑ x ∈ B, x = ∑ j ∈ J, a j := by
    calc
      ∑ x ∈ B, x = ∑ x ∈ Finset.image a J, x := by rw [h_image_eq]
      _ = ∑ j ∈ J, a j := by
        refine Finset.sum_image ?_
        intro i hi j hj h
        exact ha h
  have hprod : ∏ x ∈ B, x = ∏ j ∈ J, a j := by
    calc
```

```
      ∏ x ∈ B, x = ∏ x ∈ Finset.image a J, x := by rw [h_image_eq]
      _ = ∏ j ∈ J, a j := by
        refine Finset.prod_image ?_
        intro i hi j hj h
        exact ha h
  rcases hc J hJnon with ⟨hsum_c, hprod_c⟩
  rw [hsum, hprod]
  exact ⟨hsum_c, hprod_c⟩
```

Proof. Put $J = \{j : a_j \in B\}$.

- J is nonempty: an element of B has the form a_j, and then $j \in J$.
- $a(J) = B$: every a_j with $j \in J$ lies in B by definition, and every element of $B \subseteq a(\{0, \dots, m-1\})$ is some a_j with $j \in J$.

Since a is injective,

$$\sigma(B) = \sum_{j \in J} a_j \quad \text{and} \quad \pi(B) = \prod_{j \in J} a_j.$$

The hypothesis applied to J gives the claim. □

Lemma 3.5 (`hfs_gt_two`). *If a finite $A \subseteq \mathbb{N}_0$ satisfies (G), then $x > 2$ for every $x \in A$.*

Lean: `hfs_gt_two` — proved by AIProver, job 1, session 0

```
theorem hfs_gt_two (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x) :
    ∀ x ∈ A, 2 < x := by
  intro x hx
  by_cases hne : A.Nonempty
  · have hmin_mem : A.min' hne ∈ A := Finset.min'_mem _ hne
    have hmin_le : ∀ y ∈ A, A.min' hne ≤ y := λ y hy => Finset.min'_le A y hy
    have hfilter_empty : A.filter (fun y => y < A.min' hne) = ∅ := by
      apply Finset.eq_empty_of_forall_notMem
      intro y hy
      rcases Finset.mem_filter.mp hy with ⟨hyA, hylt⟩
      have hle := hmin_le y hyA
      linarith
    have hsum : ∑ y ∈ A.filter (fun y => y < A.min' hne), y = 0 := by
      rw [hfilter_empty]
      simp
    have hprod : ∏ y ∈ A.filter (fun y => y < A.min' hne), y = 1 := by
      rw [hfilter_empty]
      simp
    have hmin_ineq := hA (A.min' hne) hmin_mem
    rw [hsum, hprod] at hmin_ineq

    have h2_lt_min : 2 < A.min' hne := by omega
    have hle_x : A.min' hne ≤ x := hmin_le x hx
    omega
  ·
    exact absurd ⟨x, hx⟩ hne
```

Proof. Let $x \in A$. Then A is nonempty; let μ be its minimum. Then $A_{<\mu} = \varnothing$, so $\sigma(A_{<\mu}) = 0$ and $\pi(A_{<\mu}) = 1$. Condition (G) at μ gives $2(0+1) < \mu$. Hence $2 < \mu \le x$. □

Lemma 3.6 (`hfs_below`). *Let a finite $A \subseteq \mathbb{N}_0$ satisfy (G), let $B \subseteq A$ and $x \in A$, and suppose $y < x$ for all $y \in B$. Then*

$$2\big(\sigma(B) + \pi(B)\big) < x.$$

Lean: `hfs_below` — proved by AIProver, job 1, session 0

```
theorem hfs_below (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B : Finset ℕ) (hB : B ⊆ A) (x : ℕ) (hx : x ∈ A) (hlt : ∀ y ∈ B, y < x) :
    2 * (∑ y ∈ B, y + ∏ y ∈ B, y) < x := by
  have hB_filter : B ⊆ A.filter (fun y => y < x) := by
    intro y hy
    exact Finset.mem_filter.mpr ⟨hB hy, hlt y hy⟩
  have hsum_le : ∑ y ∈ B, y ≤ ∑ y ∈ A.filter (fun y => y < x), y :=
    Finset.sum_le_sum_of_subset hB_filter
  have hpos : ∀ y ∈ A, 1 ≤ y := by
    intro y hy
    have h := hA y hy
    have : 0 ≤ ∑ z ∈ A.filter (fun z => z < y), z := Nat.zero_le _
    have hprod_nonneg : 0 ≤ ∏ z ∈ A.filter (fun z => z < y), z := Nat.zero_le _
    have : 2 * (0 + 0) ≤ 2 * (∑ z ∈ A.filter (fun z => z < y), z +
      ∏ z ∈ A.filter (fun z => z < y), z) := by
      nlinarith
    nlinarith
  have hprod_le : ∏ y ∈ B, y ≤ ∏ y ∈ A.filter (fun y => y < x), y :=
    Finset.prod_le_prod_of_subset_of_one_le' hB_filter fun i hi hiB => hpos i (Finset.mem_filter.mp hi).1
  have hsum_prod_le : (∑ y ∈ B, y + ∏ y ∈ B, y) ≤
      (∑ y ∈ A.filter (fun y => y < x), y + ∏ y ∈ A.filter (fun y => y < x), y) :=
    Nat.add_le_add hsum_le hprod_le
  have h_mul_le : 2 * (∑ y ∈ B, y + ∏ y ∈ B, y) ≤
      2 * (∑ y ∈ A.filter (fun y => y < x), y + ∏ y ∈ A.filter (fun y => y < x), y) :=
    Nat.mul_le_mul_left 2 hsum_prod_le
  have h_lt_A : 2 * (∑ y ∈ A.filter (fun y => y < x), y + ∏ y ∈ A.filter (fun y => y < x), y) < x :=
    hA x hx
  exact lt_of_le_of_lt h_mul_le h_lt_A
```

Proof. By assumption $B \subseteq A_{<x}$, so $\sigma(B) \le \sigma(A_{<x})$.

Every $y \in A$ satisfies $y \ge 1$: by (G), y exceeds a natural number $2(\cdots) \ge 0$. A product of factors ≥ 1 can only grow when the index set is enlarged, so $\pi(B) \le \pi(A_{<x})$.

Hence

$$2\big(\sigma(B) + \pi(B)\big) \le 2\big(\sigma(A_{<x}) + \pi(A_{<x})\big) < x,$$

the last step being (G) at x. □

Lemma 3.7 (`hfs_le_mem`). *Let a finite $A \subseteq \mathbb{N}_0$ satisfy (G), let $B \subseteq A$ and $x \in B$. Then $x \le \sigma(B)$ and $x \le \pi(B)$.*

Lean: hfs_le_mem — proved by AIProver, job 1, session 0

```
theorem hfs_le_mem (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B : Finset ℕ) (hB : B ⊆ A) (x : ℕ) (hx : x ∈ B) :
    x ≤ ∑ y ∈ B, y ∧ x ≤ ∏ y ∈ B, y := by
  have hx_mem_A : x ∈ A := hB hx
  have hx_pos : 1 ≤ x := by
    by_contra! h
    have hx0 : x = 0 := by omega
    subst hx0
    have h0 := hA 0 hx_mem_A
    have h_empty : A.filter (fun y => y < (0 : ℕ)) = ∅ :=
      Finset.filter_false_of_mem (fun y _ => Nat.not_lt_zero y)
    rw [h_empty] at h0
    simp at h0
  constructor
  ·
    apply Finset.single_le_sum (fun i _ => Nat.zero_le _) hx
  ·
    have hpos_B : ∀ y ∈ B, 1 ≤ y := by
      intro y hy
      have hy_mem_A : y ∈ A := hB hy
      by_contra! h
      have hy0 : y = 0 := by omega
      subst hy0
      have h0 := hA 0 hy_mem_A
      have h_empty : A.filter (fun z => z < (0 : ℕ)) = ∅ :=
        Finset.filter_false_of_mem (fun z _ => Nat.not_lt_zero z)
      rw [h_empty] at h0
      simp at h0
    exact Finset.single_le_prod' (fun i hi => hpos_B i hi) hx
```

Proof. First, $0 \notin A$. Indeed $A_{<0} = \varnothing$, so (G) at 0 would give $2(0+1) < 0$, which is false. Hence all elements of B are ≥ 1.

The term x of a sum of nonnegative terms is at most the sum, so $x \leq \sigma(B)$. The factor x of a product of factors ≥ 1 is at most the product, so $x \leq \pi(B)$. □

Lemma 3.8 (hfs_disj_sum)**.** *Let a finite $A \subseteq \mathbb{N}_0$ satisfy (G), and let $B, C \subseteq A$ be disjoint with $\sigma(B) = \sigma(C)$. Then $B = C = \varnothing$.*

Lean: hfs_disj_sum — proved by AIProver, job 1, session 0

```
theorem hfs_disj_sum (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A) (hd : Disjoint B C)
    (h : ∑ y ∈ B, y = ∑ y ∈ C, y) : B = ∅ ∧ C = ∅ := by
  by_contra! hne

  have hBC_nonempty : (B ∪ C).Nonempty := by
    by_cases hBemp : B = ∅
    · have hC_ne : C ≠ ∅ := hne hBemp
      rcases Finset.nonempty_iff_ne_empty.mpr hC_ne with ⟨x, hx⟩
      exact ⟨x, Finset.mem_union_right B hx⟩
```

```
  · rcases Finset.nonempty_iff_ne_empty.mpr hBemp with ⟨x, hx⟩
    exact ⟨x, Finset.mem_union_left C hx⟩
let j := (B ∪ C).max' hBC_nonempty
have hj_mem : j ∈ B ∪ C := Finset.max'_mem _ hBC_nonempty
have hj_max : ∀ y ∈ B ∪ C, y ≤ j := fun y hy => Finset.le_max' (B ∪ C) y hy
have hjA : j ∈ A := by
  rcases Finset.mem_union.mp hj_mem with (hjB | hjC)
  · exact hB hjB
  · exact hC hjC
have hgrowth := hA j hjA
rcases Finset.mem_union.mp hj_mem with (hjB | hjC)
·
  have hsum := h
  rw [← Finset.sum_erase_add B (fun x => x) hjB] at hsum
  have hC_lt : ∀ y ∈ C, y < j := by
    intro y hyC
    have hy_mem : y ∈ B ∪ C := Finset.mem_union_right B hyC
    have hle := hj_max y hy_mem
    by_contra! hge
    have heq : y = j := Nat.le_antisymm hle hge
    have hjC : j ∈ C := by rw [← heq]; exact hyC
    exact hd.notMem_of_mem_left_finset hjB hjC
  have h_j_le_sumC : j ≤ ∑ y ∈ C, y := by omega
  have hsumC_le : ∑ y ∈ C, y ≤ ∑ y ∈ A.filter (fun y => y < j), y := by
    refine Finset.sum_le_sum_of_subset ?_
    intro y hy
    have hyC : y ∈ C := hy
    have hyA : y ∈ A := hC hyC
    have hy_lt_j : y < j := hC_lt y hyC
    exact Finset.mem_filter.mpr ⟨hyA, hy_lt_j⟩
  have h2S_lt_j : 2 * (∑ y ∈ A.filter (fun y => y < j), y) < j := by
    have hprod_nonneg : 0 ≤ ∏ y ∈ A.filter (fun y => y < j), y :=
      Finset.prod_nonneg (fun y _ => by omega)
    omega
  have hS_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < j), y :=
    Finset.sum_nonneg (fun y _ => by omega)
  omega
·
  have hsum := h
  rw [← Finset.sum_erase_add C (fun x => x) hjC] at hsum
  have hB_lt : ∀ y ∈ B, y < j := by
    intro y hyB
    have hy_mem : y ∈ B ∪ C := Finset.mem_union_left C hyB
    have hle := hj_max y hy_mem
    by_contra! hge
    have heq : y = j := Nat.le_antisymm hle hge
    have hjB : j ∈ B := by rw [← heq]; exact hyB
    exact hd.notMem_of_mem_right_finset hjC hjB
  have h_j_le_sumB : j ≤ ∑ y ∈ B, y := by omega
  have hsumB_le : ∑ y ∈ B, y ≤ ∑ y ∈ A.filter (fun y => y < j), y := by
    refine Finset.sum_le_sum_of_subset ?_
    intro y hy
    have hyB : y ∈ B := hy
    have hyA : y ∈ A := hB hyB
    have hy_lt_j : y < j := hB_lt y hyB
    exact Finset.mem_filter.mpr ⟨hyA, hy_lt_j⟩
  have h2S_lt_j : 2 * (∑ y ∈ A.filter (fun y => y < j), y) < j := by
    have hprod_nonneg : 0 ≤ ∏ y ∈ A.filter (fun y => y < j), y :=
      Finset.prod_nonneg (fun y _ => by omega)
```

```
    omega
  have hS_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < j), y :=
    Finset.sum_nonneg (fun y _ => by omega)
  omega
```

Proof. Suppose $B \cup C \neq \varnothing$ and let $j = \max(B \cup C)$, so $j \in A$. By symmetry assume $j \in B$; the case $j \in C$ is identical with the roles of B and C exchanged.

Every $y \in C$ satisfies $y \leq j$, and $y \neq j$ because B and C are disjoint. Hence $C \subseteq A_{<j}$. Writing $S = \sigma(A_{<j})$, we obtain

$$j \leq \sigma(B \setminus \{j\}) + j = \sigma(B) = \sigma(C) \leq S.$$

On the other hand, (G) at j gives $2S \leq 2(S + \pi(A_{<j})) < j$. Combining, $2S < j \leq S$, which is impossible in $\mathbb{N}_0$. □

Lemma 3.9 (hfs_disj_prod)**.** *Let a finite $A \subseteq \mathbb{N}_0$ satisfy (G), and let $B, C \subseteq A$ be disjoint with $\pi(B) = \pi(C)$. Then $B = C = \varnothing$.*

Lean: hfs_disj_prod — proved by AIProver, job 1, session 0

```
theorem hfs_disj_prod (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A) (hd : Disjoint B C)
    (h : ∏ y ∈ B, y = ∏ y ∈ C, y) : B = ∅ ∧ C = ∅ := by

  have h_gt_two : ∀ x ∈ A, 2 < x := by
    intro x hx
    have hx' := hA x hx
    have h_pos : ∀ z ∈ A, 0 < z := by
      intro z hz
      have hz' := hA z hz
      have h_nonneg : 0 ≤ 2 * (∑ y ∈ A.filter (fun y => y < z), y
        + ∏ y ∈ A.filter (fun y => y < z), y) := by omega
      omega
    have h_prod_ge_one : 1 ≤ ∏ y ∈ A.filter (fun y => y < x), y := by
      refine Finset.one_le_prod' fun y hy => ?_
      have hy_mem_A : y ∈ A := (Finset.mem_filter.mp hy).1
      have hy_pos := h_pos y hy_mem_A
      omega
    have h_sum_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < x), y :=
      Finset.sum_nonneg fun y _ => by omega
    have hineq : 2 * 1 ≤ 2 * (∑ y ∈ A.filter (fun y => y < x), y
      + ∏ y ∈ A.filter (fun y => y < x), y) := by
      nlinarith
    omega

  by_cases h_union_empty : B ∪ C = ∅
  · rcases Finset.union_eq_empty.mp h_union_empty with ⟨hB_empty, hC_empty⟩
    exact ⟨hB_empty, hC_empty⟩
  · have h_union_nonempty : (B ∪ C).Nonempty := by
      rwa [Finset.nonempty_iff_ne_empty]
    let m := (B ∪ C).max' h_union_nonempty
    have hm_mem : m ∈ B ∪ C := Finset.max'_mem _ h_union_nonempty
    have h_union_subset_A : B ∪ C ⊆ A := Finset.union_subset hB hC
    have hm_mem_A : m ∈ A := h_union_subset_A hm_mem
```

```
rcases Finset.mem_union.mp hm_mem with (hmB | hmC)
·
  have hC_lt_m : ∀ y ∈ C, y < m := by
    intro y hyC
    have hy_mem_union : y ∈ B ∪ C := Finset.mem_union_right B hyC
    have hy_le_m := Finset.le_max' (B ∪ C) y hy_mem_union
    by_contra! hge
    have heq : y = m := by omega
    have hmC' : m ∈ C := heq ▸ hyC

    exact (Finset.disjoint_left.mp hd) hmB hmC'

  have h_m_gt_2prodC : 2 * ∏ y ∈ C, y < m := by
    have hm_A := hA m hm_mem_A
    have hC_subset_filter : C ⊆ A.filter (fun y => y < m) := by
      intro y hy
      refine Finset.mem_filter.mpr ⟨hC hy, hC_lt_m y hy⟩
    have h_prod_C_le : ∏ y ∈ C, y ≤ ∏ y ∈ A.filter (fun y => y < m), y :=
      Finset.prod_le_prod_of_subset_of_one_le' hC_subset_filter fun i hi _hi_not => by
        have hi_mem_A : i ∈ A := (Finset.mem_filter.mp hi).1
        have hi_pos := h_gt_two i hi_mem_A
        omega
    have h_sum_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < m), y :=
      Finset.sum_nonneg fun y _ => by omega
    have h_2prod_lt_m : 2 * ∏ y ∈ A.filter (fun y => y < m), y < m := by
      nlinarith
    nlinarith
  have h_prod_B_ge_m : m ≤ ∏ y ∈ B, y := by
    have h_eq := Finset.mul_prod_erase B (fun y => y) hmB
    have h_rest_ge_one : 1 ≤ ∏ y ∈ B.erase m, y := by
      refine Finset.one_le_prod' fun y hy => ?_
      have hy_mem_B : y ∈ B := Finset.mem_of_mem_erase hy
      have hy_mem_A : y ∈ A := hB hy_mem_B
      have hy_gt_two := h_gt_two y hy_mem_A
      omega
    nlinarith
  have h_prod_C_ge_one : 1 ≤ ∏ y ∈ C, y := by
    refine Finset.one_le_prod' fun y hy => ?_
    have hy_mem_A : y ∈ A := hC hy
    have hy_gt_two := h_gt_two y hy_mem_A
    omega
  have h_contra : ∏ y ∈ B, y > ∏ y ∈ C, y := by
    nlinarith
  rw [h] at h_contra
  exact absurd h_contra (lt_irrefl _)
·
  have hB_lt_m : ∀ y ∈ B, y < m := by
    intro y hyB
    have hy_mem_union : y ∈ B ∪ C := Finset.mem_union_left C hyB
    have hy_le_m := Finset.le_max' (B ∪ C) y hy_mem_union
    by_contra! hge
    have heq : y = m := by omega
    have hmB' : m ∈ B := heq ▸ hyB
    exact (Finset.disjoint_left.mp hd) hmB' hmC
  have h_m_gt_2prodB : 2 * ∏ y ∈ B, y < m := by
    have hm_A := hA m hm_mem_A
    have hB_subset_filter : B ⊆ A.filter (fun y => y < m) := by
      intro y hy
```

```
        refine Finset.mem_filter.mpr ⟨hB hy, hB_lt_m y hy⟩
      have h_prod_B_le : ∏ y ∈ B, y ≤ ∏ y ∈ A.filter (fun y => y < m), y :=
        Finset.prod_le_prod_of_subset_of_one_le' hB_subset_filter fun i hi _hi_not => by
          have hi_mem_A : i ∈ A := (Finset.mem_filter.mp hi).1
          have hi_pos := h_gt_two i hi_mem_A
          omega
      have h_sum_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < m), y :=
        Finset.sum_nonneg fun y _ => by omega
      have h_2prod_lt_m : 2 * ∏ y ∈ A.filter (fun y => y < m), y < m := by
        nlinarith
      nlinarith
    have h_prod_C_ge_m : m ≤ ∏ y ∈ C, y := by
      have h_eq := Finset.mul_prod_erase C (fun y => y) hmC
      have h_rest_ge_one : 1 ≤ ∏ y ∈ C.erase m, y := by
        refine Finset.one_le_prod' fun y hy => ?_
        have hy_mem_C : y ∈ C := Finset.mem_of_mem_erase hy
        have hy_mem_A : y ∈ A := hC hy_mem_C
        have hy_gt_two := h_gt_two y hy_mem_A
        omega
      nlinarith
    have h_prod_B_ge_one : 1 ≤ ∏ y ∈ B, y := by
      refine Finset.one_le_prod' fun y hy => ?_
      have hy_mem_A : y ∈ A := hB hy
      have hy_gt_two := h_gt_two y hy_mem_A
      omega
    have h_contra : ∏ y ∈ C, y > ∏ y ∈ B, y := by
      nlinarith
    rw [← h] at h_contra
    exact absurd h_contra (lt_irrefl _)
```

Proof. First we show that every $x \in A$ exceeds 2 (as in Lemma 3.5, here derived directly). By (G), every element of A is positive. Hence $\pi(A_{<x}) \geq 1$, and (G) gives $2 \leq 2(\sigma(A_{<x}) + \pi(A_{<x})) < x$.

If $B \cup C = \varnothing$ we are done. Otherwise let $\mu = \max(B \cup C) \in A$ and assume $\mu \in B$; the case $\mu \in C$ is symmetric.

Upper bound for $\pi(C)$. As in Lemma 3.8, disjointness and maximality give $C \subseteq A_{<\mu}$. All factors are ≥ 1, so $\pi(C) \leq \pi(A_{<\mu})$. With (G) at μ this gives

$$2\pi(C) \leq 2\pi(A_{<\mu}) \leq 2\big(\sigma(A_{<\mu}) + \pi(A_{<\mu})\big) < \mu.$$

Lower bound for $\pi(B)$. We have $\pi(B) = \mu \cdot \pi(B \setminus \{\mu\})$ and $\pi(B \setminus \{\mu\}) \geq 1$. Hence $\pi(B) \geq \mu$.

Combining, $\pi(B) \geq \mu > 2\pi(C) \geq \pi(C)$, contradicting $\pi(B) = \pi(C)$. □

Lemma 3.10 (`hfs_sum_inj`). *Let a finite $A \subseteq \mathbb{N}_0$ satisfy (G), and let $B, C \subseteq A$ with $\sigma(B) = \sigma(C)$. Then $B = C$.*

Lean: `hfs_sum_inj` — proved by AIProver, job 1, session 0

```
theorem hfs_sum_inj (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A)
    (h : ∑ y ∈ B, y = ∑ y ∈ C, y) : B = C := by
  by_contra! hne

  have h_symm_nonempty : (B \ C).Nonempty ∨ (C \ B).Nonempty := by
```

```
  by_contra! hboth


  have hBC_empty : B \ C = ∅ := Finset.not_nonempty_iff_eq_empty.mp hboth.1
  have hCB_empty : C \ B = ∅ := Finset.not_nonempty_iff_eq_empty.mp hboth.2
  have hBC : B ⊆ C := by
    intro x hx
    by_contra! hx2
    have : x ∈ B \ C := Finset.mem_sdiff.mpr ⟨hx, hx2⟩
    rw [hBC_empty] at this
    simp at this
  have hCB : C ⊆ B := by
    intro x hx
    by_contra! hx2
    have : x ∈ C \ B := Finset.mem_sdiff.mpr ⟨hx, hx2⟩
    rw [hCB_empty] at this
    simp at this
  exact hne (Finset.Subset.antisymm hBC hCB)

let S := (B \ C) ∪ (C \ B)
have hS_nonempty : S.Nonempty :=
  Finset.union_nonempty.mpr h_symm_nonempty
let m := S.max' hS_nonempty
have hm_mem : m ∈ S := Finset.max'_mem _ hS_nonempty
have hmA : m ∈ A := by
  rcases Finset.mem_union.mp hm_mem with (hm' | hm')
  · exact hB (Finset.mem_sdiff.mp hm').1
  · exact hC (Finset.mem_sdiff.mp hm').1

have hineq := hA m hmA


have hS_le_m : ∀ x ∈ S, x ≤ m := by
  intro x hx
  exact Finset.le_max' S x hx

have hsum_diff : ∑ y ∈ B \ C, y = ∑ y ∈ C \ B, y := by
  have hsumB := Finset.sum_sdiff (f := id) (Finset.inter_subset_left (s₁ := B) (s₂ := C))

  have hBsdiff : B \ (B ∩ C) = B \ C := by
    ext x; simp
  have hsumC := Finset.sum_sdiff (f := id) (Finset.inter_subset_right (s₁ := B) (s₂ := C))
  have hCsdiff : C \ (B ∩ C) = C \ B := by
    ext x; simp
  rw [hBsdiff] at hsumB
  rw [hCsdiff] at hsumC
  have hsum_eq : ∑ x ∈ B \ C, id x = ∑ x ∈ C \ B, id x := by
    apply add_right_cancel
    calc
      ∑ x ∈ B \ C, id x + ∑ x ∈ B ∩ C, id x = ∑ x ∈ B, id x := hsumB
      _ = ∑ x ∈ C, id x := by simpa using h
      _ = ∑ x ∈ C \ B, id x + ∑ x ∈ B ∩ C, id x := by symm; exact hsumC
  simpa using hsum_eq

rcases Finset.mem_union.mp hm_mem with (hmB | hmC)
·


  have hCB_lt_m : ∀ y ∈ C \ B, y < m := by
```

```
    intro y hy
    have hyS : y ∈ S := Finset.mem_union_right _ hy
    have hyle := hS_le_m y hyS
    by_contra! hge
    have heq : y = m := by omega
    subst heq

    have hm_notin_B : m ∉ B := (Finset.mem_sdiff.mp hy).2
    have hm_in_B : m ∈ B := (Finset.mem_sdiff.mp hmB).1
    exact hm_notin_B hm_in_B

  have hCB_sub_A : C \ B ⊆ A := by
    intro y hy
    exact hC (Finset.mem_sdiff.mp hy).1
  have hCB_sub_filter : C \ B ⊆ A.filter (fun y => y < m) := by
    intro y hy
    apply Finset.mem_filter.mpr
    exact ⟨hCB_sub_A hy, hCB_lt_m y hy⟩

  have hsum_le : ∑ y ∈ C \ B, y ≤ ∑ y ∈ A.filter (fun y => y < m), y :=
    Finset.sum_le_sum_of_subset hCB_sub_filter


  have h2S_lt_m : 2 * (∑ y ∈ A.filter (fun y => y < m), y) < m := by
    have hpos_prod : 0 ≤ ∏ y ∈ A.filter (fun y => y < m), y :=
      Finset.prod_nonneg (fun y _ => by omega)
    omega

  have hm_le_sumBC : m ≤ ∑ y ∈ B \ C, y := by
    have := Finset.single_le_sum (fun y hy => Nat.zero_le y) hmB
    simpa using this


  rw [hsum_diff] at hm_le_sumBC
  have hm_le_S : m ≤ ∑ y ∈ A.filter (fun y => y < m), y := by omega
  have hm_pos : 0 < m := by
    by_contra! hm0
    have hm0' : m = 0 := by omega
    rw [hm0'] at hineq
    have hfilter_empty : A.filter (fun z => z < (0 : ℕ)) = ∅ :=
      Finset.filter_false _
    rw [hfilter_empty] at hineq
    simp at hineq
  omega
.


  have hBC_lt_m : ∀ y ∈ B \ C, y < m := by
    intro y hy
    have hyS : y ∈ S := Finset.mem_union_left _ hy
    have hyle := hS_le_m y hyS
    by_contra! hge
    have heq : y = m := by omega
    subst heq
    have hm_notin_C : m ∉ C := (Finset.mem_sdiff.mp hy).2
    have hm_in_C : m ∈ C := (Finset.mem_sdiff.mp hmC).1
    exact hm_notin_C hm_in_C

  have hBC_sub_A : B \ C ⊆ A := by
    intro y hy
```

```
    exact hB (Finset.mem_sdiff.mp hy).1
  have hBC_sub_filter : B \ C ⊆ A.filter (fun y => y < m) := by
    intro y hy
    apply Finset.mem_filter.mpr
    exact ⟨hBC_sub_A hy, hBC_lt_m y hy⟩

  have hsum_le : ∑ y ∈ B \ C, y ≤ ∑ y ∈ A.filter (fun y => y < m), y :=
    Finset.sum_le_sum_of_subset hBC_sub_filter

  have h2S_lt_m : 2 * (∑ y ∈ A.filter (fun y => y < m), y) < m := by
    have hpos_prod : 0 ≤ ∏ y ∈ A.filter (fun y => y < m), y :=
      Finset.prod_nonneg (fun y _ => by omega)
    omega

  have hm_le_sumCB : m ≤ ∑ y ∈ C \ B, y := by
    have := Finset.single_le_sum (fun y hy => Nat.zero_le y) hmC
    simpa using this


  rw [← hsum_diff] at hm_le_sumCB
  have hm_le_S : m ≤ ∑ y ∈ A.filter (fun y => y < m), y := by omega
  have hm_pos : 0 < m := by
    by_contra! hm0
    have hm0' : m = 0 := by omega
    rw [hm0'] at hineq
    have hfilter_empty : A.filter (fun z => z < (0 : ℕ)) = ∅ :=
      Finset.filter_false _
    rw [hfilter_empty] at hineq
    simp at hineq
  omega
```

Proof. Suppose $B \neq C$. Then $S = (B \setminus C) \cup (C \setminus B)$ is nonempty; let $\mu = \max S \in A$.

Splitting off the common part,

$$\sigma(B) = \sigma(B \setminus C) + \sigma(B \cap C), \qquad \sigma(C) = \sigma(C \setminus B) + \sigma(B \cap C).$$

Cancelling gives $\sigma(B \setminus C) = \sigma(C \setminus B)$.

Assume $\mu \in B \setminus C$; the other case is symmetric. Every $y \in C \setminus B$ lies in S, so $y \leq \mu$, and $y \neq \mu$ because $\mu \in B$. Hence $C \setminus B \subseteq A_{<\mu}$. Writing $T = \sigma(A_{<\mu})$, we obtain

$$\mu \leq \sigma(B \setminus C) = \sigma(C \setminus B) \leq T.$$

By (G) at μ and $\pi \geq 0$, $2T < \mu$. Thus $2T < \mu \leq T$, a contradiction.

(The formal proof also records $\mu > 0$, which follows from (G) at 0 as in Lemma 3.7; it is not needed for the contradiction.) □

Lemma 3.11 (`hfs_prod_inj`)**.** *Let a finite $A \subseteq \mathbb{N}_0$ satisfy (G), and let $B, C \subseteq A$ with $\pi(B) = \pi(C)$. Then $B = C$.*

Lean: `hfs_prod_inj` — proved by the captain after a hand-back

```
theorem hfs_prod_inj (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A)
```

```
    (h : ∏ y ∈ B, y = ∏ y ∈ C, y) : B = C := by
  have hD1 : B ∩ C ⊆ B := fun x hx => (Finset.mem_inter.mp hx).1
  have hD2 : B ∩ C ⊆ C := fun x hx => (Finset.mem_inter.mp hx).2
  have hpB : (∏ y ∈ B \ (B ∩ C), y) * ∏ y ∈ B ∩ C, y = ∏ y ∈ B, y :=
    Finset.prod_sdiff hD1
  have hpC : (∏ y ∈ C \ (B ∩ C), y) * ∏ y ∈ B ∩ C, y = ∏ y ∈ C, y :=
    Finset.prod_sdiff hD2
  have hpos : 0 < ∏ y ∈ B ∩ C, y := by
    apply Finset.prod_pos
    intro y hy
    have := hfs_gt_two A hA y (hB (hD1 hy))
    omega
  have heq : (∏ y ∈ B \ (B ∩ C), y) = ∏ y ∈ C \ (B ∩ C), y := by
    apply Nat.eq_of_mul_eq_mul_right hpos
    rw [hpB, hpC]
    exact h
  have hdisj : Disjoint (B \ (B ∩ C)) (C \ (B ∩ C)) := by
    rw [Finset.disjoint_left]
    intro x hx1 hx2
    rw [Finset.mem_sdiff, Finset.mem_inter] at hx1 hx2
    exact hx1.2 ⟨hx1.1, hx2.1⟩
  obtain ⟨h1, h2⟩ := hfs_disj_prod A hA _ _
    (fun x hx => hB (Finset.mem_sdiff.mp hx).1)
    (fun x hx => hC (Finset.mem_sdiff.mp hx).1) hdisj heq
  have hBC : B ⊆ C := by
    intro x hx
    by_contra hxC
    have hm : x ∈ B \ (B ∩ C) :=
      Finset.mem_sdiff.mpr ⟨hx, fun hm => hxC (Finset.mem_inter.mp hm).2⟩
    rw [h1] at hm
    exact absurd hm (Finset.notMem_empty x)
  have hCB : C ⊆ B := by
    intro x hx
    by_contra hxB
    have hm : x ∈ C \ (B ∩ C) :=
      Finset.mem_sdiff.mpr ⟨hx, fun hm => hxB (Finset.mem_inter.mp hm).1⟩
    rw [h2] at hm
    exact absurd hm (Finset.notMem_empty x)
  exact Finset.Subset.antisymm hBC hCB
```

Proof. Let $I = B \cap C$. Then

$$\pi(B) = \pi(B \setminus I)\,\pi(I), \qquad \pi(C) = \pi(C \setminus I)\,\pi(I).$$

By Lemma 3.5, every element of $I \subseteq A$ exceeds 2, so $\pi(I) > 0$. Cancelling $\pi(I)$ gives $\pi(B \setminus I) = \pi(C \setminus I)$.

The sets $B \setminus I$ and $C \setminus I$ are disjoint subsets of A: an element of both would lie in $B \cap C = I$. By Lemma 3.9, both are empty. Hence $B \subseteq C$ and $C \subseteq B$, so $B = C$. □

Lemma 3.12 (`hfs_sum_eq_prod`)**.** *Let a finite $A \subseteq \mathbb{N}_0$ satisfy (G), let $B, C \subseteq A$ be nonempty, and suppose $\sigma(B) = \pi(C)$. Then $\sigma(B) \in A$.*

Lean: `hfs_sum_eq_prod` — proved by the captain after a hand-back

```
theorem hfs_sum_eq_prod (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A) (hBne : B.Nonempty) (hCne : C.Nonempty)
    (h : ∑ y ∈ B, y = ∏ y ∈ C, y) : ∑ y ∈ B, y ∈ A := by
  have h2 := hfs_gt_two A hA
  obtain ⟨b, hbB, hbmax⟩ := Finset.exists_max_image B id hBne
  obtain ⟨c, hcC, hcmax⟩ := Finset.exists_max_image C id hCne
  have hbmax' : ∀ y ∈ B, y ≤ b := fun y hy => hbmax y hy
  have hcmax' : ∀ y ∈ C, y ≤ c := fun y hy => hcmax y hy
  have hbA : b ∈ A := hB hbB
  have hcA : c ∈ A := hC hcC
  have hbS : b ≤ ∑ y ∈ B, y := (hfs_le_mem A hA B hB b hbB).1
  have hcS : c ≤ ∏ y ∈ C, y := (hfs_le_mem A hA C hC c hcC).2
  rcases lt_trichotomy b c with hlt | heq | hgt
  · exfalso
    have hh := hfs_below A hA B hB c hcA (fun y hy => lt_of_le_of_lt (hbmax' y hy) hlt)
    omega
  · have hbC : b ∈ C := by rw [heq]; exact hcC
    have hcmax'' : ∀ y ∈ C, y ≤ b := by
      intro y hy
      have := hcmax' y hy
      omega
    have hB'sub : B.erase b ⊆ A := (Finset.erase_subset _ _).trans hB
    have hlt' : ∀ y ∈ B.erase b, y < b := fun y hy =>
      lt_of_le_of_ne (hbmax' y (Finset.mem_of_mem_erase hy)) (Finset.ne_of_mem_erase hy)
    have hbelow := hfs_below A hA (B.erase b) hB'sub b hbA hlt'
    have hsumB : ∑ y ∈ B.erase b, y + b = ∑ y ∈ B, y :=
      Finset.sum_erase_add B (fun y => y) hbB
    have hprodC : (∏ y ∈ C.erase b, y) * b = ∏ y ∈ C, y :=
      Finset.prod_erase_mul C (fun y => y) hbC
    have hP : 0 < ∏ y ∈ C.erase b, y :=
      Finset.prod_pos (fun y hy => by
        have := h2 y (hC (Finset.mem_of_mem_erase hy))
        omega)
    by_cases hP1 : ∏ y ∈ C.erase b, y = 1
    · have hSb : ∑ y ∈ B, y = b := by
        rw [h, ← hprodC, hP1, one_mul]
      rw [hSb]
      exact hbA
    · exfalso
      have hge : 2 * b ≤ (∏ y ∈ C.erase b, y) * b :=
        Nat.mul_le_mul_right b (by omega)
      omega
  · exfalso
    have hh := hfs_below A hA C hC b hbA (fun y hy => lt_of_le_of_lt (hcmax' y hy) hgt)
    omega
```

Proof. Let $b = \max B$ and $c = \max C$; both lie in A. By Lemma 3.7, $b \le \sigma(B)$ and $c \le \pi(C)$. We compare b and c.

Case $b < c$. All elements of B are $< c$. Lemma 3.6 gives $2(\sigma(B) + \pi(B)) < c \le \pi(C) = \sigma(B)$, which is impossible.

Case $b > c$. All elements of C are $< b$. Lemma 3.6 gives $2(\sigma(C) + \pi(C)) < b \le \sigma(B) = \pi(C)$, again impossible.

Case $b = c$. Then $b \in C$ and $b = \max C$. All elements of $B \setminus \{b\}$ are $< b$, so Lemma 3.6 gives

$$2\,\sigma(B \setminus \{b\}) \le 2\big(\sigma(B \setminus \{b\}) + \pi(B \setminus \{b\})\big) < b.$$

We also have

$$\sigma(B) = \sigma(B \setminus \{b\}) + b, \qquad \pi(C) = \pi(C \setminus \{b\}) \cdot b,$$

and $P = \pi(C \setminus \{b\}) > 0$ by Lemma 3.5.

- If $P = 1$, then $\sigma(B) = \pi(C) = b \in A$, as required.
- If $P \neq 1$, then $P \ge 2$, so $\sigma(B) = \pi(C) \ge 2b$. But $2\sigma(B) = 2\sigma(B \setminus \{b\}) + 2b < 3b$, so $\sigma(B) < 2b$. This is a contradiction.

□

Lemma 3.13 (`hfs_card_nonempty`)**.** *For every finite $A \subseteq \mathbb{N}_0$, the number of nonempty subsets of A is $2^{|A|} - 1$.*

Lean: `hfs_card_nonempty` — proved by AIProver, job 1, session 0

```
theorem hfs_card_nonempty (A : Finset ℕ) :
    (A.powerset.filter (fun B => B.Nonempty)).card = 2 ^ A.card - 1 := by
  have h_total : A.powerset.card = 2 ^ A.card := Finset.card_powerset _
  have h_mem : ∅ ∈ A.powerset := by
    apply Finset.mem_powerset.mpr
    exact Finset.empty_subset _
  have h_empty_not_nonempty : ¬ (∅ : Finset ℕ).Nonempty := Finset.not_nonempty_empty
  have h_filter_eq : A.powerset.filter (fun B => B.Nonempty) = A.powerset.erase ∅ := by
    ext B
    constructor
    · intro h
      have hB_mem : B ∈ A.powerset := (Finset.mem_filter.mp h).1
      have hB_nonempty : B.Nonempty := (Finset.mem_filter.mp h).2
      apply Finset.mem_erase.mpr
      constructor
      · intro h_eq; rw [h_eq] at hB_nonempty; exact Finset.not_nonempty_empty hB_nonempty
      · exact hB_mem
    · intro h
      have hB_mem : B ∈ A.powerset := (Finset.mem_erase.mp h).2
      have hB_ne : B ≠ ∅ := (Finset.mem_erase.mp h).1
      apply Finset.mem_filter.mpr
      constructor
      · exact hB_mem
      · by_contra h_not; apply hB_ne; exact (Finset.not_nonempty_iff_eq_empty.mp h_not)
  have h_card_erase : (A.powerset.erase ∅).card = A.powerset.card - 1 := by
    apply Finset.card_erase_of_mem h_mem
  rw [h_filter_eq, h_card_erase, h_total]
```

Proof. The nonempty subsets of A are the elements of the power set of A other than $\varnothing$, and $\varnothing$ belongs to the power set. The power set has $2^{|A|}$ elements, so removing $\varnothing$ leaves $2^{|A|} - 1$. □

Lemma 3.14 (`hfs_FS_card`)**.** *If a finite $A \subseteq \mathbb{N}_0$ satisfies (G), then $|\mathrm{FS}(A)| = 2^{|A|} - 1$.*

Lean: hfs_FS_card — proved by AIProver, job 1, session 0

```
theorem hfs_FS_card (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x) :
    (HindmanFSFP.FS A).card = 2 ^ A.card - 1 := by
  have hcard_subsets : (A.powerset.filter (fun B => B.Nonempty)).card = 2 ^ A.card - 1 :=
    hfs_card_nonempty A
  have h_inj_on : ∀ (B C : Finset ℕ),
    B ∈ (A.powerset.filter (fun B => B.Nonempty)) →
    C ∈ (A.powerset.filter (fun B => B.Nonempty)) →
    (∑ a ∈ B, a) = (∑ a ∈ C, a) → B = C := by
    intro B C hB hC hsum
    rcases Finset.mem_filter.mp hB with ⟨hB_pow, hB_ne⟩
    rcases Finset.mem_filter.mp hC with ⟨hC_pow, hC_ne⟩
    have hB_sub : B ⊆ A := Finset.mem_powerset.mp hB_pow
    have hC_sub : C ⊆ A := Finset.mem_powerset.mp hC_pow
    exact hfs_sum_inj A hA B C hB_sub hC_sub hsum
  have hcard_eq : (HindmanFSFP.FS A).card = (A.powerset.filter (fun B => B.Nonempty)).card := by
    rw [HindmanFSFP.FS]
    apply Finset.card_image_of_injOn
    intro B hB C hC hsum
    exact h_inj_on B C hB hC hsum
  calc
    (HindmanFSFP.FS A).card = (A.powerset.filter (fun B => B.Nonempty)).card := hcard_eq
    _ = 2 ^ A.card - 1 := hcard_subsets
```

Proof. $\mathrm{FS}(A)$ is the image of the family of nonempty subsets of A under σ. By Lemma 3.10, σ is injective on this family. The image therefore has the same cardinality as the family, which is $2^{|A|} - 1$ by Lemma 3.13. □

Lemma 3.15 (hfs_FP_card)**.** *If a finite $A \subseteq \mathbb{N}_0$ satisfies (G), then $|\mathrm{FP}(A)| = 2^{|A|} - 1$.*

Lean: hfs_FP_card — proved by AIProver, job 1, session 0

```
theorem hfs_FP_card (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x) :
    (HindmanFSFP.FP A).card = 2 ^ A.card - 1 := by
  let S := A.powerset.filter (fun B => B.Nonempty)
  have hFP_eq : HindmanFSFP.FP A = S.image (fun B => ∏ a ∈ B, a) := by
    ext x
    simp [HindmanFSFP.FP, S, Finset.mem_image, Finset.mem_filter, Finset.mem_powerset]
  rw [hFP_eq]
  have h_card_S : S.card = 2 ^ A.card - 1 := by
    have h_empty_mem : (∅ : Finset ℕ) ∈ A.powerset := by simp
    have hS_eq : S = (A.powerset).erase ∅ := by
      ext B
      simp only [S, Finset.mem_filter, Finset.mem_powerset, Finset.mem_erase]
      rw [Finset.nonempty_iff_ne_empty]
      exact and_comm
    rw [hS_eq]
    rw [Finset.card_erase_of_mem h_empty_mem, Finset.card_powerset]
  have h_inj : Set.InjOn (fun (B : Finset ℕ) => ∏ a ∈ B, a) (S : Set (Finset ℕ)) := by
    intro B hB C hC hprod
    have hB_mem : B ∈ S := hB
```

```
    have hC_mem : C ∈ S := hC
    have hB_mem_filter : B ∈ A.powerset.filter (fun B => B.Nonempty) := by simpa [S] using hB_mem
    have hC_mem_filter : C ∈ A.powerset.filter (fun B => B.Nonempty) := by simpa [S] using hC_mem
    have hB_sub : B ⊆ A := Finset.mem_powerset.mp (Finset.mem_filter.mp hB_mem_filter).left
    have hC_sub : C ⊆ A := Finset.mem_powerset.mp (Finset.mem_filter.mp hC_mem_filter).left
    exact hfs_prod_inj A hA B C hB_sub hC_sub hprod
  rw [Finset.card_image_of_injOn h_inj, h_card_S]
```

Proof. $\mathrm{FP}(A)$ is the image of the family of nonempty subsets of A under π. By Lemma 3.11, π is injective on this family. The family is the power set of A with $\varnothing$ removed, so it has $2^{|A|}-1$ elements; this count is redone inline rather than cited from Lemma 3.13. □

Lemma 3.16 (`hfs_inter`). *If a finite $A \subseteq \mathbb{N}_0$ satisfies (G), then $\mathrm{FS}(A) \cap \mathrm{FP}(A) = A$.*

Lean: `hfs_inter` — proved by AIProver, job 1, session 0

```
theorem hfs_inter (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x) :
    HindmanFSFP.FS A ∩ HindmanFSFP.FP A = A := by
  apply Finset.Subset.antisymm
  · intro x hx
    rcases Finset.mem_inter.mp hx with ⟨hxFS, hxFP⟩
    rw [setting_mem_FS] at hxFS
    rw [setting_mem_FP] at hxFP
    rcases hxFS with ⟨B, hB, hBne, hsum⟩
    rcases hxFP with ⟨C, hC, hCne, hprod⟩
    have hsum_eq_prod : ∑ y ∈ B, y = ∏ y ∈ C, y := by rw [hsum, hprod]
    have hx_in_A : ∑ y ∈ B, y ∈ A :=
      hfs_sum_eq_prod A hA B C hB hC hBne hCne hsum_eq_prod
    rw [← hsum]
    exact hx_in_A
  · intro x hx
    have hsing_nonempty : ({x} : Finset ℕ).Nonempty := by
      refine ⟨x, Finset.mem_singleton.mpr rfl⟩
    have hsum_sing : ∑ y ∈ ({x} : Finset ℕ), y = x := by simp
    have hprod_sing : ∏ y ∈ ({x} : Finset ℕ), y = x := by simp
    have hxFS : x ∈ HindmanFSFP.FS A := by
      rw [setting_mem_FS]
      refine ⟨{ x }, ?_, hsing_nonempty, hsum_sing⟩
      apply Finset.singleton_subset_iff.mpr
      exact hx
    have hxFP : x ∈ HindmanFSFP.FP A := by
      rw [setting_mem_FP]
      refine ⟨{ x }, ?_, hsing_nonempty, hprod_sing⟩
      apply Finset.singleton_subset_iff.mpr
      exact hx
    exact Finset.mem_inter.mpr ⟨hxFS, hxFP⟩
```

Proof. $\mathrm{FS}(A) \cap \mathrm{FP}(A) \subseteq A$. Let $x \in \mathrm{FS}(A) \cap \mathrm{FP}(A)$. By Lemmas 2.4 and 2.5, there are nonempty $B, C \subseteq A$ with $\sigma(B) = x = \pi(C)$. Then $x \in A$ by Lemma 3.12.

$A \subseteq \mathrm{FS}(A) \cap \mathrm{FP}(A)$. For $x \in A$, the singleton $\{x\}$ is a nonempty subset of A with $\sigma(\{x\}) = \pi(\{x\}) = x$. So $x \in \mathrm{FS}(A) \cap \mathrm{FP}(A)$ by Lemmas 2.4 and 2.5. □

3.2 Proof of the theorem

Lean: `solution` — main proof from the captain's sketch

```
theorem solution (hT : HindmanFSFP.Thm11)
    (r m : ℕ) (hr : 1 ≤ r) (hm : 1 ≤ m) (χ : ℕ → Fin r) :
    ∃ A : Finset ℕ, A.card = m ∧ (∀ a ∈ A, 0 < a) ∧
      (∃ c : Fin r, ∀ x ∈ HindmanFSFP.FS A ∪ HindmanFSFP.FP A, χ x = c) ∧
      (HindmanFSFP.FS A).card = 2 ^ m - 1 ∧ (HindmanFSFP.FP A).card = 2 ^ m - 1 ∧
      HindmanFSFP.FS A ∩ HindmanFSFP.FP A = A := by
  obtain ⟨a, c, ha, hpos, hc, hg⟩ := hfs_extract hT r m hr hm χ
  have hinj : Function.Injective a := ha.injective
  have hA := hfs_growth_image m a ha hg
  have hcard : (Finset.univ.image a).card = m := by
    rw [Finset.card_image_of_injective _ hinj, Finset.card_univ, Fintype.card_fin]
  refine ⟨Finset.univ.image a, hcard, ?_, ⟨c, ?_⟩, ?_, ?_, hfs_inter _ hA⟩
  · intro x hx
    obtain ⟨j, -, rfl⟩ := Finset.mem_image.mp hx
    exact hpos j
  · intro x hx
    rcases Finset.mem_union.mp hx with h | h
    · obtain ⟨B, hB, hne, rfl⟩ := (setting_mem_FS _ _).mp h
      exact (hfs_mono_image m r a χ c hinj hc B hB hne).1
    · obtain ⟨B, hB, hne, rfl⟩ := (setting_mem_FP _ _).mp h
      exact (hfs_mono_image m r a χ c hinj hc B hB hne).2
  · rw [hfs_FS_card _ hA, hcard]
  · rw [hfs_FP_card _ hA, hcard]
```

Proof. Assume `Thm11`, and let $r, m \geq 1$ and $\chi : \mathbb{N}_0 \to \{0, \dots, r-1\}$ be given.

By Lemma 3.2, there are a strictly increasing, positive $a : \{0, \dots, m-1\} \to \mathbb{N}_0$ and a colour c such that:

- all subset sums and subset products of the a_j have colour c;
- $2(\sum_{k<d} a_k + \prod_{k<d} a_k) < a_d$ for every d.

Let $A = \{a_0, \dots, a_{m-1}\}$ and verify the four conclusions.

1. *Size and positivity.* Since a is injective, $|A| = m$. All elements of A are positive because every $a_j > 0$.
2. *Monochromaticity.* By Lemma 3.3, A satisfies (G). Let $x \in \mathrm{FS}(A) \cup \mathrm{FP}(A)$. By Lemmas 2.4 and 2.5, $x = \sigma(B)$ or $x = \pi(B)$ for some nonempty $B \subseteq A$. In either case $\chi(x) = c$ by Lemma 3.4.
3. *Cardinalities.* By Lemmas 3.14 and 3.15 together with $|A| = m$, $|\mathrm{FS}(A)| = |\mathrm{FP}(A)| = 2^m - 1$.
4. *Intersection.* $\mathrm{FS}(A) \cap \mathrm{FP}(A) = A$ by Lemma 3.16.

□

4 Independent review

A referee model (claude-opus-5-5), which took no part in the formalization or the proof, compared the verified Lean file with the source. Verdict: **FAITHFUL**.

Definitions. `HindmanFSFP.FS` and `HindmanFSFP.FP` are the images of the nonempty members of the powerset of A under $B \mapsto \sum_{a\in B} a$ and $B \mapsto \prod_{a\in B} a$. These are exactly $\mathrm{FS}(A)$ and $\mathrm{FP}(A)$. The helper lemmas `setting_mem_FS` and `setting_mem_FP` confirm the membership characterisation.

`HindmanFSFP.Thm11` encodes Theorem 1.1, as follows.

- It quantifies over $r, m \geq 1$, real $R \geq 2$, real $D \geq 1$ and colorings $\chi : \mathbb{N} \to \operatorname{Fin} r$.
- It asserts a strictly increasing, positive $a : \operatorname{Fin} m \to \mathbb{N}$ and a colour c such that $\chi(\sum_J a) = \chi(\prod_J a) = c$ for every nonempty J.
- The growth conditions are $R < a_0$, and $R(\sum_{k<d} a_k + \prod_{k<d} a_k)^D < a_d$ (real power) for $d \geq 1$. With 0-based indexing these match $a_1 > R$ and the condition for $2 \leq d \leq m$.

Lean's $\mathbb{N}$ contains 0, so colorings here are defined on $\{0, 1, 2, \ldots\}$. This is harmless. Given such a χ, the source theorem applies to its restriction to the positive integers. All sums and products in the conclusion are positive, so the source Theorem 1.1 implies `Thm11`. The hypothesis is therefore no stronger than the source result. No discrepancy found.

Statement. The target takes `Thm11` as a hypothesis, as the source prescribes. It then assumes $r \geq 1$, $m \geq 1$ and $\chi : \mathbb{N} \to \operatorname{Fin} r$.

It concludes that there is a finset A with the following properties:

- $|A| = m$ and all elements of A are positive, so $A \subset \{1, 2, \ldots\}$;
- some colour c is taken by every element of $\mathrm{FS}(A) \cup \mathrm{FP}(A)$;
- $|\mathrm{FS}(A)| = |\mathrm{FP}(A)| = 2^m - 1$;
- $\mathrm{FS}(A) \cap \mathrm{FP}(A) = A$.

This is exactly Corollary 1.2.

A finite coloring of the positive integers extends arbitrarily to 0. The conclusion only involves positive numbers, so the source statement follows. The condition $r \geq 1$ is necessary for a coloring to exist, so it costs nothing. There is no vacuity: the definitions are non-degenerate, and the conclusions are substantive for every $m \geq 1$.

Proof. `hfs_extract` instantiates `Thm11` with $R = 2$, $D = 1$ and removes the real power via `Real.rpow_one`. It obtains $2(\sum_{k<d} a_k + \prod_{k<d} a_k) < a_d$ for all d. For $d = 0$ the empty sum and product give $2 < a_0$.

`hfs_growth_image` transfers this inequality to the set $A = \mathrm{image}(a)$, using strict monotonicity. `hfs_mono_image` transfers monochromaticity from index sets to subsets of A.

The remaining lemmas carry out the source argument:

- `hfs_sum_inj` proves injectivity of subset sums by taking the maximum m of the symmetric difference and comparing it with the sum of all smaller elements.
- `hfs_prod_inj` cancels the common part, which has positive product, and applies `hfs_disj_prod`. That lemma compares the largest remaining factor with twice the product of the smaller ones. An empty side gives 1.

- `hfs_sum_eq_prod` follows the source's case split on the greatest indices. If they differ, `hfs_below` gives a contradiction. If they are equal to b, either the remaining product is 1, so the sum equals $b \in A$, or the product is at least $2b$, exceeding the sum. The sum is less than $2b$ by `hfs_below` applied to $B \setminus \{b\}$.

The cardinalities follow from `Finset.card_image_of_injOn` and the count $2^{|A|} - 1$ of nonempty subsets. The intersection follows from `hfs_sum_eq_prod` together with the singletons. The proof matches the source proof in structure. Its only deviation is the slightly stronger intermediate inequality (factor 2) that comes from $R = 2$; this is a legitimate consequence of the hypothesis. No lemma is weaker than the step it represents.

Concerns.

- The result is conditional on `Thm11`, as the source prescribes. Its validity rests on the source paper's proof of Theorem 1.1. The Lean encoding is implied by that theorem, so the hypothesis is not an artificially strengthened assumption.

A Complete Lean file

The verified file, as checked by Lean.

```
import Mathlib
set_option autoImplicit false

namespace HindmanFSFP

/-- FS(A): the set of all sums over nonempty subsets of A. -/
def FS (A : Finset ℕ) : Finset ℕ :=
  (A.powerset.filter (fun B => B.Nonempty)).image (fun B => ∑ a ∈ B, a)

/-- FP(A): the set of all products over nonempty subsets of A. -/
def FP (A : Finset ℕ) : Finset ℕ :=
  (A.powerset.filter (fun B => B.Nonempty)).image (fun B => ∏ a ∈ B, a)

/-- Theorem 1.1 (Hindman's FS+FP conjecture with growth conditions), stated as a
proposition so that it can be used as a hypothesis. Indices run over `Fin m`, so
`a ⟨0, _⟩` plays the role of `a_1`. -/
def Thm11 : Prop :=
  ∀ (r m : ℕ), 1 ≤ r → 1 ≤ m → ∀ (R D : ℝ), 2 ≤ R → 1 ≤ D →
    ∀ χ : ℕ → Fin r, ∃ (a : Fin m → ℕ) (c : Fin r),
      StrictMono a ∧ (∀ j, 0 < a j) ∧
      (∀ J : Finset (Fin m), J.Nonempty →
        χ (∑ j ∈ J, a j) = c ∧ χ (∏ j ∈ J, a j) = c) ∧
      (∀ d : Fin m, d.val = 0 → R < (a d : ℝ)) ∧
      (∀ d : Fin m, 1 ≤ d.val →
        R * ((((∑ k ∈ Finset.Iio d, a k : ℕ) : ℝ)
              + (((∏ k ∈ Finset.Iio d, a k : ℕ) : ℝ)) ^ D) < (a d : ℝ))

end HindmanFSFP

theorem setting_mem_FS (A : Finset ℕ) (x : ℕ) :
    x ∈ HindmanFSFP.FS A ↔ ∃ B : Finset ℕ, B ⊆ A ∧ B.Nonempty ∧ ∑ a ∈ B, a = x := by
  simp only [HindmanFSFP.FS, Finset.mem_image, Finset.mem_filter, Finset.mem_powerset, and_assoc]

theorem setting_mem_FP (A : Finset ℕ) (x : ℕ) :
```

```
    x ∈ HindmanFSFP.FP A ↔ ∃ B : Finset ℕ, B ⊆ A ∧ B.Nonempty ∧ ∏ a ∈ B, a = x := by
  simp only [HindmanFSFP.FP, Finset.mem_image, Finset.mem_filter, Finset.mem_powerset, and_assoc]



theorem hfs_extract (hT : HindmanFSFP.Thm11) (r m : ℕ) (hr : 1 ≤ r) (hm : 1 ≤ m)
    (χ : ℕ → Fin r) :
    ∃ (a : Fin m → ℕ) (c : Fin r), StrictMono a ∧ (∀ j, 0 < a j) ∧
      (∀ J : Finset (Fin m), J.Nonempty → χ (∑ j ∈ J, a j) = c ∧ χ (∏ j ∈ J, a j) = c) ∧
      (∀ d : Fin m, 2 * (∑ k ∈ Finset.Iio d, a k + ∏ k ∈ Finset.Iio d, a k) < a d) := by

  have h := hT r m hr hm (2 : ℝ) (1 : ℝ) (by norm_num) (by norm_num) χ
  rcases h with ⟨a, c, hmono, hpos, hcolor, hfirst, hrest⟩
  refine ⟨a, c, hmono, hpos, hcolor, ?_⟩
  intro d
  by_cases hd0 : d.val = 0
  ·
    have hineq : (2 : ℝ) < (a d : ℝ) := hfirst d hd0
    have hempty : Finset.Iio d = (∅ : Finset (Fin m)) := by
      apply Finset.eq_empty_iff_forall_notMem.mpr
      intro x hx
      have hxval := Finset.mem_Iio.mp hx
      have hxval_lt : x.val < d.val := hxval
      rw [hd0] at hxval_lt
      exact Nat.not_lt_zero _ hxval_lt
    have hsum : (∑ k ∈ Finset.Iio d, a k) = 0 := by simp [hempty]
    have hprod : (∏ k ∈ Finset.Iio d, a k) = 1 := by simp [hempty]
    rw [hsum, hprod]

    have hcast : ((2 : ℕ) : ℝ) < (a d : ℝ) := by simpa using hineq

    simpa [Nat.cast_add, Nat.cast_mul, Nat.cast_one] using (Nat.cast_lt (α := ℝ)).mp hcast
  ·
    have hdpos : 1 ≤ d.val := Nat.one_le_of_lt (Nat.pos_of_ne_zero hd0)
    have hineq : (2 : ℝ) * (((∑ k ∈ Finset.Iio d, a k : ℕ) : ℝ) + ((∏ k ∈ Finset.Iio d, a k : ℕ) : ℝ))
↪   ^ (1 : ℝ) < (a d : ℝ) :=
      hrest d hdpos

    have hineq' : (2 : ℝ) * (((∑ k ∈ Finset.Iio d, a k : ℕ) : ℝ) + ((∏ k ∈ Finset.Iio d, a k : ℕ) : ℝ))
↪   < (a d : ℝ) := by
      simpa [Real.rpow_one] using hineq

    have hcast : ((2 * ((∑ k ∈ Finset.Iio d, a k) + (∏ k ∈ Finset.Iio d, a k)) : ℕ) : ℝ) < (a d : ℝ) := by
      simpa [Nat.cast_add, Nat.cast_mul] using hineq'
    exact (Nat.cast_lt (α := ℝ)).mp hcast

theorem hfs_growth_image (m : ℕ) (a : Fin m → ℕ) (ha : StrictMono a)
    (hg : ∀ d : Fin m, 2 * (∑ k ∈ Finset.Iio d, a k + ∏ k ∈ Finset.Iio d, a k) < a d) :
    ∀ x ∈ Finset.univ.image a,
      2 * (∑ y ∈ (Finset.univ.image a).filter (fun y => y < x), y
        + ∏ y ∈ (Finset.univ.image a).filter (fun y => y < x), y) < x := by
  intro x hx
  rcases Finset.mem_image.1 hx with ⟨d, _, hd⟩
  subst hd
  have h_filter_eq : (Finset.univ.image a).filter (fun y => y < a d) =
      (Finset.Iio d).image a := by
    apply Finset.Subset.antisymm
    · intro y hy
      rcases Finset.mem_filter.1 hy with ⟨hy_img, hy_lt⟩
```

```
      rcases Finset.mem_image.1 hy_img with ⟨i, _, hi⟩
      have hi_lt_d : i < d := by
        by_contra! hge
        have hle := ha.monotone hge
        rw [hi] at hle
        linarith
      apply Finset.mem_image.mpr
      exact ⟨i, Finset.mem_Iio.mpr hi_lt_d, hi⟩
    · intro y hy
      rcases Finset.mem_image.1 hy with ⟨i, hi, rfl⟩
      have hi_lt_d : i < d := Finset.mem_Iio.1 hi
      have ha_lt : a i < a d := ha hi_lt_d
      apply Finset.mem_filter.mpr
      exact ⟨Finset.mem_image.mpr ⟨i, Finset.mem_univ _, rfl⟩, ha_lt⟩
  have h_sum_eq : ∑ y ∈ (Finset.univ.image a).filter (fun y => y < a d), y =
      ∑ k ∈ Finset.Iio d, a k := by
    rw [h_filter_eq, Finset.sum_image]
    intro i _ j _ h
    apply ha.injective h
  have h_prod_eq : ∏ y ∈ (Finset.univ.image a).filter (fun y => y < a d), y =
      ∏ k ∈ Finset.Iio d, a k := by
    rw [h_filter_eq, Finset.prod_image]
    intro i _ j _ h
    apply ha.injective h
  rw [h_sum_eq, h_prod_eq]
  exact hg d

theorem hfs_mono_image (m r : ℕ) (a : Fin m → ℕ) (χ : ℕ → Fin r) (c : Fin r)
    (ha : Function.Injective a)
    (hc : ∀ J : Finset (Fin m), J.Nonempty → χ (∑ j ∈ J, a j) = c ∧ χ (∏ j ∈ J, a j) = c) :
    ∀ B ⊆ Finset.univ.image a, B.Nonempty → χ (∑ x ∈ B, x) = c ∧ χ (∏ x ∈ B, x) = c := by
  intro B hBsub hBnon

  let J := Finset.univ.filter fun j : Fin m => a j ∈ B
  have hJnon : J.Nonempty := by
    rcases hBnon with ⟨x, hx⟩
    have hx_image : x ∈ Finset.univ.image a := hBsub hx
    rcases Finset.mem_image.mp hx_image with ⟨j, hj, rfl⟩
    refine ⟨j, Finset.mem_filter.mpr ⟨Finset.mem_univ _, hx⟩⟩
  have h_image_eq : Finset.image a J = B := by
    apply Finset.Subset.antisymm
    · intro x hx
      rcases Finset.mem_image.mp hx with ⟨j, hj, rfl⟩
      rcases Finset.mem_filter.mp hj with ⟨_, hjB⟩
      exact hjB
    · intro x hx
      have hx_image : x ∈ Finset.univ.image a := hBsub hx
      rcases Finset.mem_image.mp hx_image with ⟨j, hj, rfl⟩
      apply Finset.mem_image.mpr
      exact ⟨j, Finset.mem_filter.mpr ⟨Finset.mem_univ _, hx⟩, rfl⟩
  have hsum : ∑ x ∈ B, x = ∑ j ∈ J, a j := by
    calc
      ∑ x ∈ B, x = ∑ x ∈ Finset.image a J, x := by rw [h_image_eq]
      _ = ∑ j ∈ J, a j := by
        refine Finset.sum_image ?_
        intro i hi j hj h
        exact ha h
  have hprod : ∏ x ∈ B, x = ∏ j ∈ J, a j := by
    calc
```

```
      ∏ x ∈ B, x = ∏ x ∈ Finset.image a J, x := by rw [h_image_eq]
      _ = ∏ j ∈ J, a j := by
        refine Finset.prod_image ?_
        intro i hi j hj h
        exact ha h
  rcases hc J hJnon with ⟨hsum_c, hprod_c⟩
  rw [hsum, hprod]
  exact ⟨hsum_c, hprod_c⟩

theorem hfs_gt_two (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x) :
    ∀ x ∈ A, 2 < x := by
  intro x hx
  by_cases hne : A.Nonempty
  · have hmin_mem : A.min' hne ∈ A := Finset.min'_mem _ hne
    have hmin_le : ∀ y ∈ A, A.min' hne ≤ y := λ y hy => Finset.min'_le A y hy
    have hfilter_empty : A.filter (fun y => y < A.min' hne) = ∅ := by
      apply Finset.eq_empty_of_forall_notMem
      intro y hy
      rcases Finset.mem_filter.mp hy with ⟨hyA, hylt⟩
      have hle := hmin_le y hyA
      linarith
    have hsum : ∑ y ∈ A.filter (fun y => y < A.min' hne), y = 0 := by
      rw [hfilter_empty]
      simp
    have hprod : ∏ y ∈ A.filter (fun y => y < A.min' hne), y = 1 := by
      rw [hfilter_empty]
      simp
    have hmin_ineq := hA (A.min' hne) hmin_mem
    rw [hsum, hprod] at hmin_ineq

    have h2_lt_min : 2 < A.min' hne := by omega
    have hle_x : A.min' hne ≤ x := hmin_le x hx
    omega
  ·
    exact absurd ⟨x, hx⟩ hne

theorem hfs_below (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B : Finset ℕ) (hB : B ⊆ A) (x : ℕ) (hx : x ∈ A) (hlt : ∀ y ∈ B, y < x) :
    2 * (∑ y ∈ B, y + ∏ y ∈ B, y) < x := by
  have hB_filter : B ⊆ A.filter (fun y => y < x) := by
    intro y hy
    exact Finset.mem_filter.mpr ⟨hB hy, hlt y hy⟩
  have hsum_le : ∑ y ∈ B, y ≤ ∑ y ∈ A.filter (fun y => y < x), y :=
    Finset.sum_le_sum_of_subset hB_filter
  have hpos : ∀ y ∈ A, 1 ≤ y := by
    intro y hy
    have h := hA y hy
    have : 0 ≤ ∑ z ∈ A.filter (fun z => z < y), z := Nat.zero_le _
    have hprod_nonneg : 0 ≤ ∏ z ∈ A.filter (fun z => z < y), z := Nat.zero_le _
    have : 2 * (0 + 0) ≤ 2 * (∑ z ∈ A.filter (fun z => z < y), z +
      ∏ z ∈ A.filter (fun z => z < y), z) := by
      nlinarith
    nlinarith
  have hprod_le : ∏ y ∈ B, y ≤ ∏ y ∈ A.filter (fun y => y < x), y :=
    Finset.prod_le_prod_of_subset_of_one_le' hB_filter fun i hi hiB => hpos i (Finset.mem_filter.mp hi).1
```

```
  have hsum_prod_le : (∑ y ∈ B, y + ∏ y ∈ B, y) ≤
      (∑ y ∈ A.filter (fun y => y < x), y + ∏ y ∈ A.filter (fun y => y < x), y) :=
    Nat.add_le_add hsum_le hprod_le
  have h_mul_le : 2 * (∑ y ∈ B, y + ∏ y ∈ B, y) ≤
      2 * (∑ y ∈ A.filter (fun y => y < x), y + ∏ y ∈ A.filter (fun y => y < x), y) :=
    Nat.mul_le_mul_left 2 hsum_prod_le
  have h_lt_A : 2 * (∑ y ∈ A.filter (fun y => y < x), y + ∏ y ∈ A.filter (fun y => y < x), y) < x :=
    hA x hx
  exact lt_of_le_of_lt h_mul_le h_lt_A

theorem hfs_le_mem (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B : Finset ℕ) (hB : B ⊆ A) (x : ℕ) (hx : x ∈ B) :
    x ≤ ∑ y ∈ B, y ∧ x ≤ ∏ y ∈ B, y := by
  have hx_mem_A : x ∈ A := hB hx
  have hx_pos : 1 ≤ x := by
    by_contra! h
    have hx0 : x = 0 := by omega
    subst hx0
    have h0 := hA 0 hx_mem_A
    have h_empty : A.filter (fun y => y < (0 : ℕ)) = ∅ :=
      Finset.filter_false_of_mem (fun y _ => Nat.not_lt_zero y)
    rw [h_empty] at h0
    simp at h0
  constructor
  ·
    apply Finset.single_le_sum (fun i _ => Nat.zero_le _) hx
  ·
    have hpos_B : ∀ y ∈ B, 1 ≤ y := by
      intro y hy
      have hy_mem_A : y ∈ A := hB hy
      by_contra! h
      have hy0 : y = 0 := by omega
      subst hy0
      have h0 := hA 0 hy_mem_A
      have h_empty : A.filter (fun z => z < (0 : ℕ)) = ∅ :=
        Finset.filter_false_of_mem (fun z _ => Nat.not_lt_zero z)
      rw [h_empty] at h0
      simp at h0
    exact Finset.single_le_prod' (fun i hi => hpos_B i hi) hx

theorem hfs_disj_sum (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A) (hd : Disjoint B C)
    (h : ∑ y ∈ B, y = ∑ y ∈ C, y) : B = ∅ ∧ C = ∅ := by
  by_contra! hne

  have hBC_nonempty : (B ∪ C).Nonempty := by
    by_cases hBemp : B = ∅
    · have hC_ne : C ≠ ∅ := hne hBemp
      rcases Finset.nonempty_iff_ne_empty.mpr hC_ne with ⟨x, hx⟩
      exact ⟨x, Finset.mem_union_right B hx⟩
    · rcases Finset.nonempty_iff_ne_empty.mpr hBemp with ⟨x, hx⟩
      exact ⟨x, Finset.mem_union_left C hx⟩
  let j := (B ∪ C).max' hBC_nonempty
  have hj_mem : j ∈ B ∪ C := Finset.max'_mem _ hBC_nonempty
  have hj_max : ∀ y ∈ B ∪ C, y ≤ j := fun y hy => Finset.le_max' (B ∪ C) y hy
```

```
have hjA : j ∈ A := by
  rcases Finset.mem_union.mp hj_mem with (hjB | hjC)
  · exact hB hjB
  · exact hC hjC
have hgrowth := hA j hjA
rcases Finset.mem_union.mp hj_mem with (hjB | hjC)
·
  have hsum := h
  rw [← Finset.sum_erase_add B (fun x => x) hjB] at hsum
  have hC_lt : ∀ y ∈ C, y < j := by
    intro y hyC
    have hy_mem : y ∈ B ∪ C := Finset.mem_union_right B hyC
    have hle := hj_max y hy_mem
    by_contra! hge
    have heq : y = j := Nat.le_antisymm hle hge
    have hjC : j ∈ C := by rw [← heq]; exact hyC
    exact hd.notMem_of_mem_left_finset hjB hjC
  have h_j_le_sumC : j ≤ ∑ y ∈ C, y := by omega
  have hsumC_le : ∑ y ∈ C, y ≤ ∑ y ∈ A.filter (fun y => y < j), y := by
    refine Finset.sum_le_sum_of_subset ?_
    intro y hy
    have hyC : y ∈ C := hy
    have hyA : y ∈ A := hC hyC
    have hy_lt_j : y < j := hC_lt y hyC
    exact Finset.mem_filter.mpr ⟨hyA, hy_lt_j⟩
  have h2S_lt_j : 2 * (∑ y ∈ A.filter (fun y => y < j), y) < j := by
    have hprod_nonneg : 0 ≤ ∏ y ∈ A.filter (fun y => y < j), y :=
      Finset.prod_nonneg (fun y _ => by omega)
    omega
  have hS_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < j), y :=
    Finset.sum_nonneg (fun y _ => by omega)
  omega
·
  have hsum := h
  rw [← Finset.sum_erase_add C (fun x => x) hjC] at hsum
  have hB_lt : ∀ y ∈ B, y < j := by
    intro y hyB
    have hy_mem : y ∈ B ∪ C := Finset.mem_union_left C hyB
    have hle := hj_max y hy_mem
    by_contra! hge
    have heq : y = j := Nat.le_antisymm hle hge
    have hjB : j ∈ B := by rw [← heq]; exact hyB
    exact hd.notMem_of_mem_right_finset hjC hjB
  have h_j_le_sumB : j ≤ ∑ y ∈ B, y := by omega
  have hsumB_le : ∑ y ∈ B, y ≤ ∑ y ∈ A.filter (fun y => y < j), y := by
    refine Finset.sum_le_sum_of_subset ?_
    intro y hy
    have hyB : y ∈ B := hy
    have hyA : y ∈ A := hB hyB
    have hy_lt_j : y < j := hB_lt y hyB
    exact Finset.mem_filter.mpr ⟨hyA, hy_lt_j⟩
  have h2S_lt_j : 2 * (∑ y ∈ A.filter (fun y => y < j), y) < j := by
    have hprod_nonneg : 0 ≤ ∏ y ∈ A.filter (fun y => y < j), y :=
      Finset.prod_nonneg (fun y _ => by omega)
    omega
  have hS_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < j), y :=
    Finset.sum_nonneg (fun y _ => by omega)
  omega
```

```
theorem hfs_disj_prod (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A) (hd : Disjoint B C)
    (h : ∏ y ∈ B, y = ∏ y ∈ C, y) : B = ∅ ∧ C = ∅ := by

  have h_gt_two : ∀ x ∈ A, 2 < x := by
    intro x hx
    have hx' := hA x hx
    have h_pos : ∀ z ∈ A, 0 < z := by
      intro z hz
      have hz' := hA z hz
      have h_nonneg : 0 ≤ 2 * (∑ y ∈ A.filter (fun y => y < z), y
        + ∏ y ∈ A.filter (fun y => y < z), y) := by omega
      omega
    have h_prod_ge_one : 1 ≤ ∏ y ∈ A.filter (fun y => y < x), y := by
      refine Finset.one_le_prod' fun y hy => ?_
      have hy_mem_A : y ∈ A := (Finset.mem_filter.mp hy).1
      have hy_pos := h_pos y hy_mem_A
      omega
    have h_sum_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < x), y :=
      Finset.sum_nonneg fun y _ => by omega
    have hineq : 2 * 1 ≤ 2 * (∑ y ∈ A.filter (fun y => y < x), y
      + ∏ y ∈ A.filter (fun y => y < x), y) := by
      nlinarith
    omega

  by_cases h_union_empty : B ∪ C = ∅
  · rcases Finset.union_eq_empty.mp h_union_empty with ⟨hB_empty, hC_empty⟩
    exact ⟨hB_empty, hC_empty⟩
  · have h_union_nonempty : (B ∪ C).Nonempty := by
      rwa [Finset.nonempty_iff_ne_empty]
    let m := (B ∪ C).max' h_union_nonempty
    have hm_mem : m ∈ B ∪ C := Finset.max'_mem _ h_union_nonempty
    have h_union_subset_A : B ∪ C ⊆ A := Finset.union_subset hB hC
    have hm_mem_A : m ∈ A := h_union_subset_A hm_mem

    rcases Finset.mem_union.mp hm_mem with (hmB | hmC)
    ·
      have hC_lt_m : ∀ y ∈ C, y < m := by
        intro y hyC
        have hy_mem_union : y ∈ B ∪ C := Finset.mem_union_right B hyC
        have hy_le_m := Finset.le_max' (B ∪ C) y hy_mem_union
        by_contra! hge
        have heq : y = m := by omega
        have hmC' : m ∈ C := heq ▸ hyC

        exact (Finset.disjoint_left.mp hd) hmB hmC'

      have h_m_gt_2prodC : 2 * ∏ y ∈ C, y < m := by
        have hm_A := hA m hm_mem_A
        have hC_subset_filter : C ⊆ A.filter (fun y => y < m) := by
          intro y hy
          refine Finset.mem_filter.mpr ⟨hC hy, hC_lt_m y hy⟩
        have h_prod_C_le : ∏ y ∈ C, y ≤ ∏ y ∈ A.filter (fun y => y < m), y :=
          Finset.prod_le_prod_of_subset_of_one_le' hC_subset_filter fun i hi _hi_not => by
            have hi_mem_A : i ∈ A := (Finset.mem_filter.mp hi).1
            have hi_pos := h_gt_two i hi_mem_A
            omega
```

```
    have h_sum_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < m), y :=
      Finset.sum_nonneg fun y _ => by omega
    have h_2prod_lt_m : 2 * ∏ y ∈ A.filter (fun y => y < m), y < m := by
      nlinarith
    nlinarith
  have h_prod_B_ge_m : m ≤ ∏ y ∈ B, y := by
    have h_eq := Finset.mul_prod_erase B (fun y => y) hmB
    have h_rest_ge_one : 1 ≤ ∏ y ∈ B.erase m, y := by
      refine Finset.one_le_prod' fun y hy => ?_
      have hy_mem_B : y ∈ B := Finset.mem_of_mem_erase hy
      have hy_mem_A : y ∈ A := hB hy_mem_B
      have hy_gt_two := h_gt_two y hy_mem_A
      omega
    nlinarith
  have h_prod_C_ge_one : 1 ≤ ∏ y ∈ C, y := by
    refine Finset.one_le_prod' fun y hy => ?_
    have hy_mem_A : y ∈ A := hC hy
    have hy_gt_two := h_gt_two y hy_mem_A
    omega
  have h_contra : ∏ y ∈ B, y > ∏ y ∈ C, y := by
    nlinarith
  rw [h] at h_contra
  exact absurd h_contra (lt_irrefl _)
·
  have hB_lt_m : ∀ y ∈ B, y < m := by
    intro y hyB
    have hy_mem_union : y ∈ B ∪ C := Finset.mem_union_left C hyB
    have hy_le_m := Finset.le_max' (B ∪ C) y hy_mem_union
    by_contra! hge
    have heq : y = m := by omega
    have hmB' : m ∈ B := heq ▸ hyB
    exact (Finset.disjoint_left.mp hd) hmB' hmC
  have h_m_gt_2prodB : 2 * ∏ y ∈ B, y < m := by
    have hm_A := hA m hm_mem_A
    have hB_subset_filter : B ⊆ A.filter (fun y => y < m) := by
      intro y hy
      refine Finset.mem_filter.mpr ⟨hB hy, hB_lt_m y hy⟩
    have h_prod_B_le : ∏ y ∈ B, y ≤ ∏ y ∈ A.filter (fun y => y < m), y :=
      Finset.prod_le_prod_of_subset_of_one_le' hB_subset_filter fun i hi _hi_not => by
        have hi_mem_A : i ∈ A := (Finset.mem_filter.mp hi).1
        have hi_pos := h_gt_two i hi_mem_A
        omega
    have h_sum_nonneg : 0 ≤ ∑ y ∈ A.filter (fun y => y < m), y :=
      Finset.sum_nonneg fun y _ => by omega
    have h_2prod_lt_m : 2 * ∏ y ∈ A.filter (fun y => y < m), y < m := by
      nlinarith
    nlinarith
  have h_prod_C_ge_m : m ≤ ∏ y ∈ C, y := by
    have h_eq := Finset.mul_prod_erase C (fun y => y) hmC
    have h_rest_ge_one : 1 ≤ ∏ y ∈ C.erase m, y := by
      refine Finset.one_le_prod' fun y hy => ?_
      have hy_mem_C : y ∈ C := Finset.mem_of_mem_erase hy
      have hy_mem_A : y ∈ A := hC hy_mem_C
      have hy_gt_two := h_gt_two y hy_mem_A
      omega
    nlinarith
  have h_prod_B_ge_one : 1 ≤ ∏ y ∈ B, y := by
    refine Finset.one_le_prod' fun y hy => ?_
    have hy_mem_A : y ∈ A := hB hy
```

```
        have hy_gt_two := h_gt_two y hy_mem_A
        omega
      have h_contra : ∏ y ∈ C, y > ∏ y ∈ B, y := by
        nlinarith
      rw [← h] at h_contra
      exact absurd h_contra (lt_irrefl _)

theorem hfs_sum_inj (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A)
    (h : ∑ y ∈ B, y = ∑ y ∈ C, y) : B = C := by
  by_contra! hne

  have h_symm_nonempty : (B \ C).Nonempty ∨ (C \ B).Nonempty := by
    by_contra! hboth


    have hBC_empty : B \ C = ∅ := Finset.not_nonempty_iff_eq_empty.mp hboth.1
    have hCB_empty : C \ B = ∅ := Finset.not_nonempty_iff_eq_empty.mp hboth.2
    have hBC : B ⊆ C := by
      intro x hx
      by_contra! hx2
      have : x ∈ B \ C := Finset.mem_sdiff.mpr ⟨hx, hx2⟩
      rw [hBC_empty] at this
      simp at this
    have hCB : C ⊆ B := by
      intro x hx
      by_contra! hx2
      have : x ∈ C \ B := Finset.mem_sdiff.mpr ⟨hx, hx2⟩
      rw [hCB_empty] at this
      simp at this
    exact hne (Finset.Subset.antisymm hBC hCB)

  let S := (B \ C) ∪ (C \ B)
  have hS_nonempty : S.Nonempty :=
    Finset.union_nonempty.mpr h_symm_nonempty
  let m := S.max' hS_nonempty
  have hm_mem : m ∈ S := Finset.max'_mem _ hS_nonempty
  have hmA : m ∈ A := by
    rcases Finset.mem_union.mp hm_mem with (hm' | hm')
    · exact hB (Finset.mem_sdiff.mp hm').1
    · exact hC (Finset.mem_sdiff.mp hm').1

  have hineq := hA m hmA


  have hS_le_m : ∀ x ∈ S, x ≤ m := by
    intro x hx
    exact Finset.le_max' S x hx

  have hsum_diff : ∑ y ∈ B \ C, y = ∑ y ∈ C \ B, y := by
    have hsumB := Finset.sum_sdiff (f := id) (Finset.inter_subset_left (s₁ := B) (s₂ := C))

    have hBsdiff : B \ (B ∩ C) = B \ C := by
      ext x; simp
    have hsumC := Finset.sum_sdiff (f := id) (Finset.inter_subset_right (s₁ := B) (s₂ := C))
    have hCsdiff : C \ (B ∩ C) = C \ B := by
      ext x; simp
```

```
  rw [hBsdiff] at hsumB
  rw [hCsdiff] at hsumC
  have hsum_eq : ∑ x ∈ B \ C, id x = ∑ x ∈ C \ B, id x := by
    apply add_right_cancel
    calc
      ∑ x ∈ B \ C, id x + ∑ x ∈ B ∩ C, id x = ∑ x ∈ B, id x := hsumB
      _ = ∑ x ∈ C, id x := by simpa using h
      _ = ∑ x ∈ C \ B, id x + ∑ x ∈ B ∩ C, id x := by symm; exact hsumC
  simpa using hsum_eq

rcases Finset.mem_union.mp hm_mem with (hmB | hmC)
.


  have hCB_lt_m : ∀ y ∈ C \ B, y < m := by
    intro y hy
    have hyS : y ∈ S := Finset.mem_union_right _ hy
    have hyle := hS_le_m y hyS
    by_contra! hge
    have heq : y = m := by omega
    subst heq

    have hm_notin_B : m ∉ B := (Finset.mem_sdiff.mp hy).2
    have hm_in_B : m ∈ B := (Finset.mem_sdiff.mp hmB).1
    exact hm_notin_B hm_in_B

  have hCB_sub_A : C \ B ⊆ A := by
    intro y hy
    exact hC (Finset.mem_sdiff.mp hy).1
  have hCB_sub_filter : C \ B ⊆ A.filter (fun y => y < m) := by
    intro y hy
    apply Finset.mem_filter.mpr
    exact ⟨hCB_sub_A hy, hCB_lt_m y hy⟩

  have hsum_le : ∑ y ∈ C \ B, y ≤ ∑ y ∈ A.filter (fun y => y < m), y :=
    Finset.sum_le_sum_of_subset hCB_sub_filter


  have h2S_lt_m : 2 * (∑ y ∈ A.filter (fun y => y < m), y) < m := by
    have hpos_prod : 0 ≤ ∏ y ∈ A.filter (fun y => y < m), y :=
      Finset.prod_nonneg (fun y _ => by omega)
    omega

  have hm_le_sumBC : m ≤ ∑ y ∈ B \ C, y := by
    have := Finset.single_le_sum (fun y hy => Nat.zero_le y) hmB
    simpa using this


  rw [hsum_diff] at hm_le_sumBC
  have hm_le_S : m ≤ ∑ y ∈ A.filter (fun y => y < m), y := by omega
  have hm_pos : 0 < m := by
    by_contra! hm0
    have hm0' : m = 0 := by omega
    rw [hm0'] at hineq
    have hfilter_empty : A.filter (fun z => z < (0 : ℕ)) = ∅ :=
      Finset.filter_false _
    rw [hfilter_empty] at hineq
    simp at hineq
  omega
```

```
  .

    have hBC_lt_m : ∀ y ∈ B \ C, y < m := by
      intro y hy
      have hyS : y ∈ S := Finset.mem_union_left _ hy
      have hyle := hS_le_m y hyS
      by_contra! hge
      have heq : y = m := by omega
      subst heq
      have hm_notin_C : m ∉ C := (Finset.mem_sdiff.mp hy).2
      have hm_in_C : m ∈ C := (Finset.mem_sdiff.mp hmC).1
      exact hm_notin_C hm_in_C

    have hBC_sub_A : B \ C ⊆ A := by
      intro y hy
      exact hB (Finset.mem_sdiff.mp hy).1
    have hBC_sub_filter : B \ C ⊆ A.filter (fun y => y < m) := by
      intro y hy
      apply Finset.mem_filter.mpr
      exact ⟨hBC_sub_A hy, hBC_lt_m y hy⟩

    have hsum_le : ∑ y ∈ B \ C, y ≤ ∑ y ∈ A.filter (fun y => y < m), y :=
      Finset.sum_le_sum_of_subset hBC_sub_filter

    have h2S_lt_m : 2 * (∑ y ∈ A.filter (fun y => y < m), y) < m := by
      have hpos_prod : 0 ≤ ∏ y ∈ A.filter (fun y => y < m), y :=
        Finset.prod_nonneg (fun y _ => by omega)
      omega

    have hm_le_sumCB : m ≤ ∑ y ∈ C \ B, y := by
      have := Finset.single_le_sum (fun y hy => Nat.zero_le y) hmC
      simpa using this


    rw [← hsum_diff] at hm_le_sumCB
    have hm_le_S : m ≤ ∑ y ∈ A.filter (fun y => y < m), y := by omega
    have hm_pos : 0 < m := by
      by_contra! hm0
      have hm0' : m = 0 := by omega
      rw [hm0'] at hineq
      have hfilter_empty : A.filter (fun z => z < (0 : ℕ)) = ∅ :=
        Finset.filter_false _
      rw [hfilter_empty] at hineq
      simp at hineq
    omega

theorem hfs_prod_inj (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A)
    (h : ∏ y ∈ B, y = ∏ y ∈ C, y) : B = C := by
  have hD1 : B ∩ C ⊆ B := fun x hx => (Finset.mem_inter.mp hx).1
  have hD2 : B ∩ C ⊆ C := fun x hx => (Finset.mem_inter.mp hx).2
  have hpB : (∏ y ∈ B \ (B ∩ C), y) * ∏ y ∈ B ∩ C, y = ∏ y ∈ B, y :=
    Finset.prod_sdiff hD1
  have hpC : (∏ y ∈ C \ (B ∩ C), y) * ∏ y ∈ B ∩ C, y = ∏ y ∈ C, y :=
    Finset.prod_sdiff hD2
  have hpos : 0 < ∏ y ∈ B ∩ C, y := by
    apply Finset.prod_pos
```

```
    intro y hy
    have := hfs_gt_two A hA y (hB (hD1 hy))
    omega
  have heq : (∏ y ∈ B \ (B ∩ C), y) = ∏ y ∈ C \ (B ∩ C), y := by
    apply Nat.eq_of_mul_eq_mul_right hpos
    rw [hpB, hpC]
    exact h
  have hdisj : Disjoint (B \ (B ∩ C)) (C \ (B ∩ C)) := by
    rw [Finset.disjoint_left]
    intro x hx1 hx2
    rw [Finset.mem_sdiff, Finset.mem_inter] at hx1 hx2
    exact hx1.2 ⟨hx1.1, hx2.1⟩
  obtain ⟨h1, h2⟩ := hfs_disj_prod A hA _ _
    (fun x hx => hB (Finset.mem_sdiff.mp hx).1)
    (fun x hx => hC (Finset.mem_sdiff.mp hx).1) hdisj heq
  have hBC : B ⊆ C := by
    intro x hx
    by_contra hxC
    have hm : x ∈ B \ (B ∩ C) :=
      Finset.mem_sdiff.mpr ⟨hx, fun hm => hxC (Finset.mem_inter.mp hm).2⟩
    rw [h1] at hm
    exact absurd hm (Finset.notMem_empty x)
  have hCB : C ⊆ B := by
    intro x hx
    by_contra hxB
    have hm : x ∈ C \ (B ∩ C) :=
      Finset.mem_sdiff.mpr ⟨hx, fun hm => hxB (Finset.mem_inter.mp hm).1⟩
    rw [h2] at hm
    exact absurd hm (Finset.notMem_empty x)
  exact Finset.Subset.antisymm hBC hCB

theorem hfs_sum_eq_prod (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x)
    (B C : Finset ℕ) (hB : B ⊆ A) (hC : C ⊆ A) (hBne : B.Nonempty) (hCne : C.Nonempty)
    (h : ∑ y ∈ B, y = ∏ y ∈ C, y) : ∑ y ∈ B, y ∈ A := by
  have h2 := hfs_gt_two A hA
  obtain ⟨b, hbB, hbmax⟩ := Finset.exists_max_image B id hBne
  obtain ⟨c, hcC, hcmax⟩ := Finset.exists_max_image C id hCne
  have hbmax' : ∀ y ∈ B, y ≤ b := fun y hy => hbmax y hy
  have hcmax' : ∀ y ∈ C, y ≤ c := fun y hy => hcmax y hy
  have hbA : b ∈ A := hB hbB
  have hcA : c ∈ A := hC hcC
  have hbS : b ≤ ∑ y ∈ B, y := (hfs_le_mem A hA B hB b hbB).1
  have hcS : c ≤ ∏ y ∈ C, y := (hfs_le_mem A hA C hC c hcC).2
  rcases lt_trichotomy b c with hlt | heq | hgt
  · exfalso
    have hh := hfs_below A hA B hB c hcA (fun y hy => lt_of_le_of_lt (hbmax' y hy) hlt)
    omega
  · have hbC : b ∈ C := by rw [heq]; exact hcC
    have hcmax'' : ∀ y ∈ C, y ≤ b := by
      intro y hy
      have := hcmax' y hy
      omega
    have hB'sub : B.erase b ⊆ A := (Finset.erase_subset _ _).trans hB
    have hlt' : ∀ y ∈ B.erase b, y < b := fun y hy =>
      lt_of_le_of_ne (hbmax' y (Finset.mem_of_mem_erase hy)) (Finset.ne_of_mem_erase hy)
    have hbelow := hfs_below A hA (B.erase b) hB'sub b hbA hlt'
    have hsumB : ∑ y ∈ B.erase b, y + b = ∑ y ∈ B, y :=
```

```
      Finset.sum_erase_add B (fun y => y) hbB
    have hprodC : (∏ y ∈ C.erase b, y) * b = ∏ y ∈ C, y :=
      Finset.prod_erase_mul C (fun y => y) hbC
    have hP : 0 < ∏ y ∈ C.erase b, y :=
      Finset.prod_pos (fun y hy => by
        have := h2 y (hC (Finset.mem_of_mem_erase hy))
        omega)
    by_cases hP1 : ∏ y ∈ C.erase b, y = 1
    · have hSb : ∑ y ∈ B, y = b := by
        rw [h, ← hprodC, hP1, one_mul]
      rw [hSb]
      exact hbA
    · exfalso
      have hge : 2 * b ≤ (∏ y ∈ C.erase b, y) * b :=
        Nat.mul_le_mul_right b (by omega)
      omega
  · exfalso
    have hh := hfs_below A hA C hC b hbA (fun y hy => lt_of_le_of_lt (hcmax' y hy) hgt)
    omega

theorem hfs_card_nonempty (A : Finset ℕ) :
    (A.powerset.filter (fun B => B.Nonempty)).card = 2 ^ A.card - 1 := by
  have h_total : A.powerset.card = 2 ^ A.card := Finset.card_powerset _
  have h_mem : ∅ ∈ A.powerset := by
    apply Finset.mem_powerset.mpr
    exact Finset.empty_subset _
  have h_empty_not_nonempty : ¬ (∅ : Finset ℕ).Nonempty := Finset.not_nonempty_empty
  have h_filter_eq : A.powerset.filter (fun B => B.Nonempty) = A.powerset.erase ∅ := by
    ext B
    constructor
    · intro h
      have hB_mem : B ∈ A.powerset := (Finset.mem_filter.mp h).1
      have hB_nonempty : B.Nonempty := (Finset.mem_filter.mp h).2
      apply Finset.mem_erase.mpr
      constructor
      · intro h_eq; rw [h_eq] at hB_nonempty; exact Finset.not_nonempty_empty hB_nonempty
      · exact hB_mem
    · intro h
      have hB_mem : B ∈ A.powerset := (Finset.mem_erase.mp h).2
      have hB_ne : B ≠ ∅ := (Finset.mem_erase.mp h).1
      apply Finset.mem_filter.mpr
      constructor
      · exact hB_mem
      · by_contra h_not; apply hB_ne; exact (Finset.not_nonempty_iff_eq_empty.mp h_not)
  have h_card_erase : (A.powerset.erase ∅).card = A.powerset.card - 1 := by
    apply Finset.card_erase_of_mem h_mem
  rw [h_filter_eq, h_card_erase, h_total]

theorem hfs_FS_card (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x) :
    (HindmanFSFP.FS A).card = 2 ^ A.card - 1 := by
  have hcard_subsets : (A.powerset.filter (fun B => B.Nonempty)).card = 2 ^ A.card - 1 :=
    hfs_card_nonempty A
  have h_inj_on : ∀ (B C : Finset ℕ),
    B ∈ (A.powerset.filter (fun B => B.Nonempty)) →
    C ∈ (A.powerset.filter (fun B => B.Nonempty)) →
    (∑ a ∈ B, a) = (∑ a ∈ C, a) → B = C := by
    intro B C hB hC hsum
```

```
    rcases Finset.mem_filter.mp hB with ⟨hB_pow, hB_ne⟩
    rcases Finset.mem_filter.mp hC with ⟨hC_pow, hC_ne⟩
    have hB_sub : B ⊆ A := Finset.mem_powerset.mp hB_pow
    have hC_sub : C ⊆ A := Finset.mem_powerset.mp hC_pow
    exact hfs_sum_inj A hA B C hB_sub hC_sub hsum
  have hcard_eq : (HindmanFSFP.FS A).card = (A.powerset.filter (fun B => B.Nonempty)).card := by
    rw [HindmanFSFP.FS]
    apply Finset.card_image_of_injOn
    intro B hB C hC hsum
    exact h_inj_on B C hB hC hsum
  calc
    (HindmanFSFP.FS A).card = (A.powerset.filter (fun B => B.Nonempty)).card := hcard_eq
    _ = 2 ^ A.card - 1 := hcard_subsets

theorem hfs_FP_card (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x) :
    (HindmanFSFP.FP A).card = 2 ^ A.card - 1 := by
  let S := A.powerset.filter (fun B => B.Nonempty)
  have hFP_eq : HindmanFSFP.FP A = S.image (fun B => ∏ a ∈ B, a) := by
    ext x
    simp [HindmanFSFP.FP, S, Finset.mem_image, Finset.mem_filter, Finset.mem_powerset]
  rw [hFP_eq]
  have h_card_S : S.card = 2 ^ A.card - 1 := by
    have h_empty_mem : (∅ : Finset ℕ) ∈ A.powerset := by simp
    have hS_eq : S = (A.powerset).erase ∅ := by
      ext B
      simp only [S, Finset.mem_filter, Finset.mem_powerset, Finset.mem_erase]
      rw [Finset.nonempty_iff_ne_empty]
      exact and_comm
    rw [hS_eq]
    rw [Finset.card_erase_of_mem h_empty_mem, Finset.card_powerset]
  have h_inj : Set.InjOn (fun (B : Finset ℕ) => ∏ a ∈ B, a) (S : Set (Finset ℕ)) := by
    intro B hB C hC hprod
    have hB_mem : B ∈ S := hB
    have hC_mem : C ∈ S := hC
    have hB_mem_filter : B ∈ A.powerset.filter (fun B => B.Nonempty) := by simpa [S] using hB_mem
    have hC_mem_filter : C ∈ A.powerset.filter (fun B => B.Nonempty) := by simpa [S] using hC_mem
    have hB_sub : B ⊆ A := Finset.mem_powerset.mp (Finset.mem_filter.mp hB_mem_filter).left
    have hC_sub : C ⊆ A := Finset.mem_powerset.mp (Finset.mem_filter.mp hC_mem_filter).left
    exact hfs_prod_inj A hA B C hB_sub hC_sub hprod
  rw [Finset.card_image_of_injOn h_inj, h_card_S]

theorem hfs_inter (A : Finset ℕ)
    (hA : ∀ x ∈ A, 2 * (∑ y ∈ A.filter (fun y => y < x), y
        + ∏ y ∈ A.filter (fun y => y < x), y) < x) :
    HindmanFSFP.FS A ∩ HindmanFSFP.FP A = A := by
  apply Finset.Subset.antisymm
  · intro x hx
    rcases Finset.mem_inter.mp hx with ⟨hxFS, hxFP⟩
    rw [setting_mem_FS] at hxFS
    rw [setting_mem_FP] at hxFP
    rcases hxFS with ⟨B, hB, hBne, hsum⟩
    rcases hxFP with ⟨C, hC, hCne, hprod⟩
    have hsum_eq_prod : ∑ y ∈ B, y = ∏ y ∈ C, y := by rw [hsum, hprod]
    have hx_in_A : ∑ y ∈ B, y ∈ A :=
      hfs_sum_eq_prod A hA B C hB hC hBne hCne hsum_eq_prod
    rw [← hsum]
    exact hx_in_A
```

```
  · intro x hx
    have hsing_nonempty : ({x} : Finset ℕ).Nonempty := by
      refine ⟨x, Finset.mem_singleton.mpr rfl⟩
    have hsum_sing : ∑ y ∈ ({x} : Finset ℕ), y = x := by simp
    have hprod_sing : ∏ y ∈ ({x} : Finset ℕ), y = x := by simp
    have hxFS : x ∈ HindmanFSFP.FS A := by
      rw [setting_mem_FS]
      refine ⟨{ x }, ?_, hsing_nonempty, hsum_sing⟩
      apply Finset.singleton_subset_iff.mpr
      exact hx
    have hxFP : x ∈ HindmanFSFP.FP A := by
      rw [setting_mem_FP]
      refine ⟨{ x }, ?_, hsing_nonempty, hprod_sing⟩
      apply Finset.singleton_subset_iff.mpr
      exact hx
    exact Finset.mem_inter.mpr ⟨hxFS, hxFP⟩

theorem solution (hT : HindmanFSFP.Thm11)
    (r m : ℕ) (hr : 1 ≤ r) (hm : 1 ≤ m) (χ : ℕ → Fin r) :
    ∃ A : Finset ℕ, A.card = m ∧ (∀ a ∈ A, 0 < a) ∧
      (∃ c : Fin r, ∀ x ∈ HindmanFSFP.FS A ∪ HindmanFSFP.FP A, χ x = c) ∧
      (HindmanFSFP.FS A).card = 2 ^ m - 1 ∧ (HindmanFSFP.FP A).card = 2 ^ m - 1 ∧
      HindmanFSFP.FS A ∩ HindmanFSFP.FP A = A := by
  obtain ⟨a, c, ha, hpos, hc, hg⟩ := hfs_extract hT r m hr hm χ
  have hinj : Function.Injective a := ha.injective
  have hA := hfs_growth_image m a ha hg
  have hcard : (Finset.univ.image a).card = m := by
    rw [Finset.card_image_of_injective _ hinj, Finset.card_univ, Fintype.card_fin]
  refine ⟨Finset.univ.image a, hcard, ?_, ⟨c, ?_⟩, ?_, ?_, hfs_inter _ hA⟩
  · intro x hx
    obtain ⟨j, -, rfl⟩ := Finset.mem_image.mp hx
    exact hpos j
  · intro x hx
    rcases Finset.mem_union.mp hx with h | h
    · obtain ⟨B, hB, hne, rfl⟩ := (setting_mem_FS _ _).mp h
      exact (hfs_mono_image m r a χ c hinj hc B hB hne).1
    · obtain ⟨B, hB, hne, rfl⟩ := (setting_mem_FP _ _).mp h
      exact (hfs_mono_image m r a χ c hinj hc B hB hne).2
  · rw [hfs_FS_card _ hA, hcard]
  · rw [hfs_FP_card _ hA, hcard]
```